

amuguna

개인 프로필 기반 공공 금융정보 매칭 서비스

제출 목적	공공데이터포털(data.go.kr) 오픈API 활용신청 및 구현 증빙
서비스명	amuguna (아무거나)
출품 대회	2026 금융 AI Challenge — 금융보안원 주최 / 금융위원회 후원 / 데이콘 운영
활용 유형	비영리 · 공모전 출품용 웹서비스 (개인/팀 개발)
기술 스택	TypeScript (Node.js 배치 + Next.js 웹) / PostgreSQL + pgvector
소스 저장소	github.com/ofseo03/amuguna
담당자 연락처	ofseo03@gmail.com
작성일	2026-08-11

본 문서는 신청 API를 실제로 어떻게 호출하고 처리하는지를 코드와 데이터 계약으로 제시하기 위해 작성되었습니다. 수집기 소스코드 전문과 API 응답 처리 구조, 호출량 산정 근거를 포함합니다.

1. 활용 목적

정부·지자체·공공기관이 제공하는 지원금·정책자금·금융상품 정보는 부처별 사이트에 흩어져 있고, 자격요건은 "만 19~34세, 중위소득 150% 이하, 무주택, 해당 시군 6개월 이상 거주" 같은 조합형입니다. 그 결과 받을 수 있는데 몰라서 신청하지 못하는 사각지대가 발생합니다.

본 서비스는 이용자가 입력한 **인적사항 5필드**(연령·성별·직업·거주지·소득분위)와 **"원하는 것" 한 줄**을 받아, 두 축의 교집합으로 정보를 좁힙니다.

축	질문	입력	판정 방식
자격	내가 대상인가?	인적사항 5필드	정형 조건 대조 (SQL)
의도	내가 찾는 것인가?	원하는 것 한 줄	의미 유사도 (벡터 검색)

자격만 맞추면 수백 건의 무관한 정보가 쏟아지고, 의도만 맞추면 대상이 아닌 것을 권하게 됩니다. 두 축이 서로 다른 것을 측정하기 때문에 **교집합이 곧 정보**가 됩니다.

공공데이터 활용의 핵심

신청 API는 이 서비스의 **유일한 정보 원천**입니다. 임의로 생성하거나 추정한 정보는 노출하지 않으며, 모든 결과 카드에 **원문 출처 URL과 수집 시각**을 함께 표시하여 이용자가 원본을 직접 확인할 수 있게 합니다.

2. 활용신청 대상 오픈API

공공데이터포털 인증키 1개로 아래 데이터셋을 활용합니다.

2.1 지원금·복지 (자격요건 원천)

데이터셋 ID	명칭	활용 내용
15113968	행정안전부_대한민국 공공서비스(혜택) 정보	중앙부처·지자체·공공기관·교육청 혜택 전체의 목록과 상세를 수집해 매칭 대상 모집단을 구성
15090532	한국사회보장정보원_중앙부처복지서비스	목록조회 뒤 신규 항목의 상세조회를 호출해 지원대상·선정기준·신청방법 텍스트에서 자격요건을 추출
15108347	한국사회보장정보원_지자체복지서비스	지자체 단위 복지서비스. 거주지 기준 필터링의 원천

2.2 서민·정책금융 상품 (대회 주제 직결)

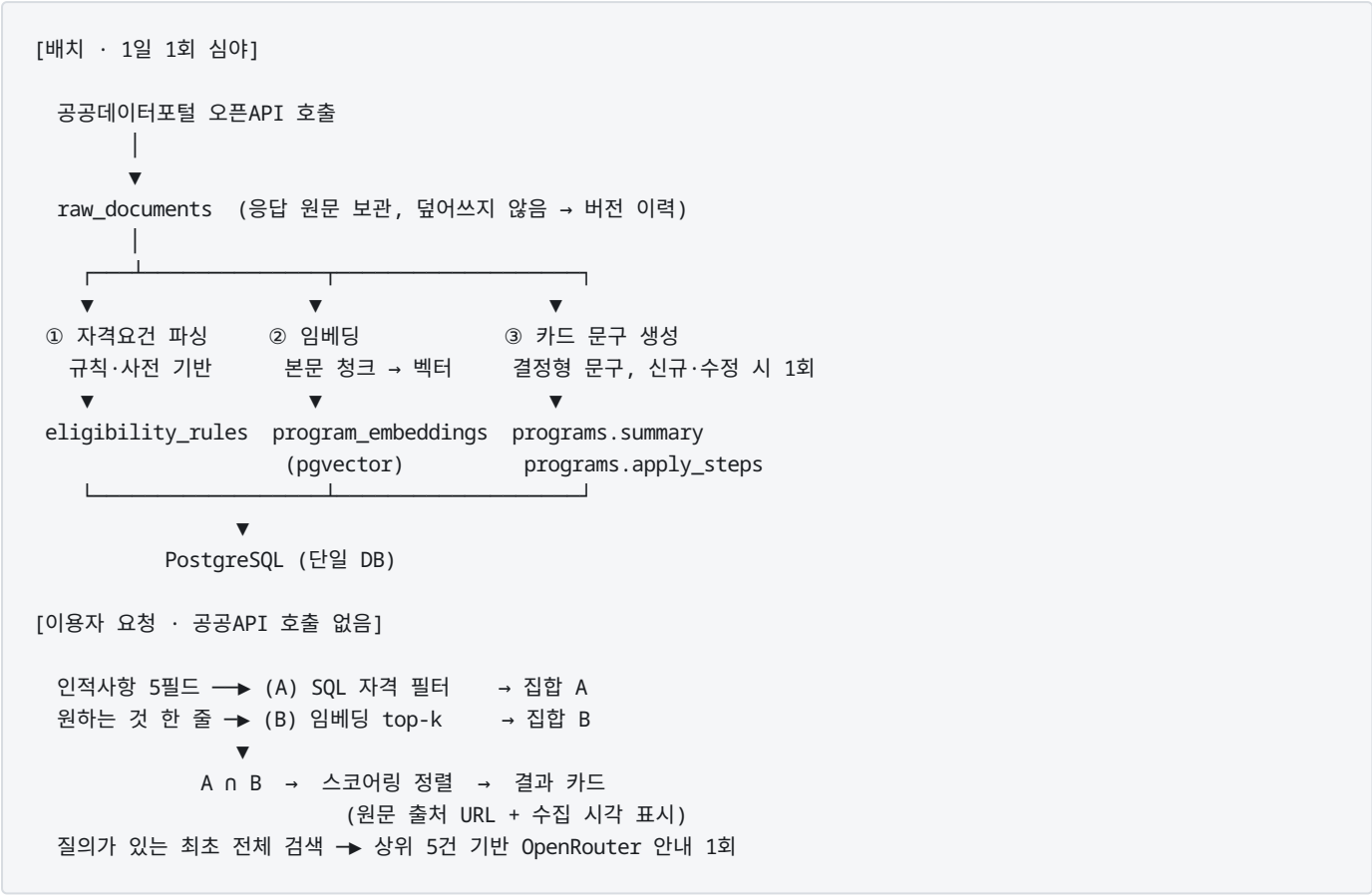
데이터셋 ID	명칭	활용 내용
15094787	금융위원회_서민금융상품기본정보	서민금융 대출·자산형성·사회적금융 상품의 지원대상·금리·한도·상환방식을 상품 카드로 제공
15106208	서민금융진흥원_대출상품한눈에 정보 서비스	자금용도·지원대상 기준 상품 매칭
15119396	한국주택금융공사_전세자금보증상품 추천서비스	전세보증금 관련 이용자에게 보증상품·지역별 최대임차보증금액 안내
15082039	한국주택금융공사_u-보증자리론대출정보	만기별 금리 정보
15074508	서민금융진흥원_서민대출상품 취급기관 정보	상품에서 실제 신청 창구로 연결
15037733	서민금융진흥원_서민금융통합지원센터 현황	디지털 취약계층 이용자에게 가까운 오프라인 창구를 함께 안내

2.3 확장

데이터셋 ID	명칭	활용 내용
15143273	한국고용정보원_온통청년_청년정책API	청년 대상 정책 보강

15125364	창업진흥원_K-Startup 조회서비스	창업·소상공인 지원사업 보강
15098547	한국부동산원_청약홈 분양정보 조회 서비스	주거 관련 공고 보강
15000115	법제처 국가법령정보 공유서비스	제도의 근거 법령 원문 링크

3. 시스템 구성 및 데이터 흐름



이용자 요청 시점에는 공공데이터 API를 호출하지 않습니다.

모든 공고 조회는 사전 수집된 데이터베이스만 대상으로 합니다. 질의가 있는 최초 전체 검색에서만 검색어와 상위 5건의 공개 정보를 OpenRouter에 보내 안내 답변을 1회 생성하며, 프로필·쿠키는 보내지 않습니다. OpenRouter 호출이 실패해도 결과 카드는 그대로 제공합니다.

3.1 증분 수집 — 호출량 절감 설계

소스가 수정일 파라미터를 지원하면 전일 이후 변경분만 조회합니다. 지원하지 않으면 content_hash 비교로 변경분만 재처리합니다.

상황	판정	처리
external_id 없음	신규	원문 저장 → 파싱 → 임베딩 → 등록
있음 + 해시 동일	무변경	수집시각만 갱신. 추가 처리 없음
있음 + 해시 상이	수정	원문 이력 추가 → 재파싱 → 재임베딩 → 갱신
전량 대조에서 누락	종료	expired 처리 (행 삭제 없음)

`content_hash` 는 응답 전체가 아니라 **비교 대상 필드만**(제목·본문·기간·금액)으로 계산합니다. 조회수나 응답 생성 시각처럼 매 호출 바뀌는 휘발성 필드를 해시에 포함하면 전 건이 매일 "수정"으로 판정되어 증분 수집의 이점이 사라지기 때문입니다.

3.2 호출량 산정 (개발계정 한도 내 누적 적재)

구분	산정
수집 주기	1일 1회 (심야 배치, GitHub Actions cron)
목록 조회	중앙부처복지는 1회 500건 · 페이지네이션
상세 조회	중앙부처복지 신규 건만 건당 1회, 회차당 최대 90회
주 1회 전량 대조	목록조회로 external ID를 비교 (건별 상세 호출 없음)
요청 간 간격	별도 1초 throttle은 현재 구현하지 않음; 페이지 요청은 순차 실행
재시도	408/429/5xx에 한해 최대 2회, 지수 백오프 (250ms → 500ms)

중앙부처복지서비스 상세조회 개발계정은 100회/일이므로, 현재 수집기는 회차당 상세 호출을 `maxDetailCalls` (기본 90)에서 멈춥니다. 이미 적재된 `external_id` 는 상세를 건너뛰어 신규 건에 예산을 쓰고, 429 등으로 중단돼도 그때까지 수집한 건은 저장해 다음 회차에 이어받습니다. 목록 API에는 날짜 필터가 없어 `--since` 를 중앙부처복지 요청에 보내지 않습니다.

4. 수집기 소스코드

공공데이터포털 API를 직접 호출하는 코드 전문입니다. 언어는 TypeScript(Node.js)입니다.

4.1 공통 수집기 — 호출·재시도·응답 검증

모든 수집기가 상속하는 기반 클래스입니다. 인증키 주입, 타임아웃, 재시도 정책, 응답 봉투(`resultCode`) 검증, 페이지네이션 종료 조건을 담당합니다. User-Agent에 서비스명과 연락처를 명시하여 포털 측에서 호출 주체를 식별할 수 있게 했습니다.

web/ingest/collectors/base.ts

308 lines · TypeScript

```
1 import { readFile } from "node:fs/promises";
2 import { resolve } from "node:path";
3
4 import { FIXTURES_DIR, settingsFromEnv, type Settings } from "../config";
5 import type { CollectedProgram } from "../models";
6
7 export const USER_AGENT =
8   "amuguna-ingest/0.1 (contact: ofseo03@gmail.com)";
9
10 type CollectorErrorOptions = ErrorOptions & { status?: number; code?: string };
11
12 export class CollectorError extends Error {
13   readonly status?: number;
14   readonly code?: string;
15
16   constructor(message: string, options?: CollectorErrorOptions) {
17     super(message, options);
18     this.name = "CollectorError";
19     this.status = options?.status;
20     this.code = options?.code;
21   }
22 }
23
24 export type FetchLike = (input: string | URL | Request, init?: RequestInit) => Promise<Response>;
25
26 export interface CollectorOptions {
27   settings?: Settings;
28   useFixtures?: boolean;
29   fetchImpl?: FetchLike;
30   pageSize?: number;
31   retries?: number;
32   timeoutMs?: number;
33   maxDetailCalls?: number;
34 }
35
36 export interface FetchOptions {
37   since?: string | null;
38   maxPages?: number;
39   /**
40    * 이미 적재된 external_id. 건당 상세 조회가 필요한 소스만 참고해, 일일 호출
41    * 한도를 신규 건에 몰아준다. 상세 호출이 없는 소스는 그대로 무시한다.
42    */
43   knownIds?: ReadonlySet<string>;
44 }
45
46 export type CollectorConstructor = new (options?: CollectorOptions) => Collector;
47
48 /** 총 건수가 없는 API의 무한 페이지를 막는다. 도달 시 성공으로 가장하지 않는다. */
49 export const PAGINATION_SAFETY_LIMIT = 1_000;
50
```

```

51 export abstract class Collector {
52     abstract readonly sourceKey: string;
53     abstract readonly endpoint: string;
54     abstract readonly idListEndpoint: string;
55     readonly defaultForm: string = "subsidy";
56     readonly incrementalStrategy: string | null = null;
57
58     readonly settings: Settings;
59     readonly useFixtures: boolean;
60     readonly pageSize: number;
61     readonly retries: number;
62     readonly timeoutMs: number;
63     httpCalls = 0;
64     observedCount: number | null = null;
65     readonly errors: string[] = [];
66
67     private readonly fetchImpl: FetchLike;
68
69     constructor(options: CollectorOptions = {}) {
70         this.settings = options.settings ?? settingsFromEnv();
71         this.useFixtures = options.useFixtures ?? false;
72         this.fetchImpl = options.fetchImpl ?? fetch;
73         this.pageSize = options.pageSize ?? 100;
74         this.retries = options.retries ?? 2;
75         this.timeoutMs = options.timeoutMs ?? 30_000;
76     }
77
78     protected abstract queryParams(options: {
79         since: string | null;
80         page: number;
81     }): Record<string, string | number>;
82
83     protected abstract items(payload: unknown): Record<string, unknown>[];
84
85     protected abstract mapItem(item: Record<string, unknown>): CollectedProgram | null;
86
87     get fixturePath(): string {
88         return resolve(FIXTURES_DIR, `${this.sourceKey}.json`);
89     }
90
91     private async loadJson(path: string): Promise<unknown> {
92         try {
93             return JSON.parse(await readFile(path, "utf8")) as unknown;
94         } catch (error) {
95             throw new CollectorError(`JSON 파일을 읽을 수 없음: ${path}`, { cause: error });
96         }
97     }
98
99     protected requireApiKey(name: string, value: string): string {
100         if (!value) throw new CollectorError(`${this.sourceKey}: ${name} 미설정`);
101         return value;
102     }
103
104     protected async request<T>(
105         params: Record<string, string | number>,
106         parse: (response: Response) => Promise<T>,
107         endpoint = this.endpoint,
108     ): Promise<T> {
109         const url = new URL(endpoint);
110         for (const [key, value] of Object.entries(params)) url.searchParams.set(key, String(value));
111
112         let lastError: unknown;
113         for (let attempt = 0; attempt <= this.retries; attempt += 1) {
114             this.httpCalls += 1;
115             try {
116                 const response = await this.fetchImpl(url, {
117                     headers: { "User-Agent": USER_AGENT },

```



```

118         redirect: "follow",
119         signal: AbortSignal.timeout(this.timeoutMs),
120     });
121     if (!response.ok) {
122         const retryable = response.status === 408 || response.status === 429 || response.status >= 500;
123         if (!retryable || attempt === this.retries) {
124             throw new CollectorError(`${this.sourceKey} 호출 실패: HTTP ${response.status}`, {
125                 status: response.status,
126             });
127         }
128         lastError = new Error(`HTTP ${response.status}`);
129     } else {
130         return await parse(response);
131     }
132 } catch (error) {
133     if (error instanceof CollectorError) throw error;
134     lastError = error;
135     if (attempt === this.retries) break;
136 }
137
138 await new Promise((done) => setTimeout(done, 250 * 2 ** attempt));
139 }
140
141 throw new CollectorError(`${this.sourceKey} 호출 실패`, { cause: lastError });
142 }
143
144 private async getJson(params: Record<string, string | number>): Promise<unknown> {
145     return this.request(params, async (response) => (await response.json()) as unknown);
146 }
147
148 externalId(nativeId: string): string {
149     return `${this.sourceKey}:${nativeId}`;
150 }
151
152 private validateEmptyPage(payload: unknown): void {
153     if (!isRecord(payload)) {
154         throw new CollectorError(`${this.sourceKey}: 오류 또는 잘못된 응답 봉투`);
155     }
156
157     let valid = false;
158     let message = "";
159
160     if (this.sourceKey === "social_security") {
161         const response = isRecord(payload.response) ? payload.response : payload;
162         const header = recordOrUndefined(response.header);
163         const body = recordOrUndefined(response.body);
164         const pageItems = body?.items;
165         valid =
166             header !== undefined &&
167             String(header.resultCode) === "00" &&
168             (Array.isArray(pageItems) ||
169              (isRecord(pageItems) &&
170               (Object.keys(pageItems).length === 0 || Object.hasOwn(pageItems, "item"))));
171         message = header ? String(header.resultMsg || "") : "";
172     } else if (this.sourceKey === "bizinfo") {
173         const response = recordOrUndefined(payload.response) ?? {};
174         const header = recordOrUndefined(response.header) ?? {};
175         const body = recordOrUndefined(response.body) ?? {};
176         const pageItems = Object.hasOwn(payload, "jsonArray") ? payload.jsonArray : body.items;
177         const code = payload.resultCode ?? header.resultCode;
178         valid = String(code) === "0000" && (Array.isArray(pageItems) || isRecord(pageItems));
179         message = String(payload.resultMsg || header.resultMsg || "");
180     } else if (this.sourceKey === "finlife") {
181         const result = recordOrUndefined(payload.result);
182         valid = result !== undefined && String(result.err_cd) === "000" && Array.isArray(result.baseList);
183         message = result ? String(result.err_msg || "") : "";
184     } else if (this.sourceKey === "gov24") {

```

```

185     valid = Array.isArray(payload.data);
186 } else if (this.sourceKey === "kstartup") {
187     valid = Array.isArray(payload.data);
188 } else {
189     return;
190 }
191
192 if (!valid) {
193     throw new CollectorError(
194         `${this.sourceKey}: 오류 또는 잘못된 응답 봉투${message ? ` `: `${message}`}` : ""}`,
195     );
196 }
197 }
198
199 async fetch(options: FetchOptions = {}): Promise<CollectedProgram[]> {
200     this.observedCount = null;
201     this.errors.length = 0;
202     const { since = null } = options;
203     const maxPages = options.maxPages ?? PAGINATION_SAFETY_LIMIT;
204     const payloads: unknown[] = [];
205     if (this.useFixtures) {
206         payloads.push(await this.loadJson(this.fixturePath));
207     } else {
208         let complete = false;
209         for (let page = 1; page <= maxPages; page += 1) {
210             const payload = await this.getJson(this.queryParams({ since, page }));
211             const pageItems = this.items(payload);
212             if (pageItems.length === 0) this.validateEmptyPage(payload);
213             payloads.push(payload);
214             if (pageItems.length < this.pageSize) {
215                 complete = true;
216                 break;
217             }
218         }
219         if (!complete && options.maxPages === undefined) {
220             throw new CollectorError(
221                 `${this.sourceKey}: ${PAGINATION_SAFETY_LIMIT}페이지 안전 한도에 도달해 전량 수집 여부를 확인할 수 없음`,
222             );
223         }
224     }
225
226     const collected: CollectedProgram[] = [];
227     const seen = new Set<string>();
228     this.observedCount = payloads.reduce<number>(
229         (count, payload) => count + this.items(payload).length,
230         0,
231     );
232     for (const payload of payloads) {
233         for (const item of this.items(payload)) {
234             try {
235                 const program = this.mapItem(item);
236                 if (!program || seen.has(program.external_id)) continue;
237                 seen.add(program.external_id);
238                 collected.push(program);
239             } catch (error) {
240                 console.warn(`${this.sourceKey}: 항목 매핑 실패`, error);
241                 this.errors.push(`${this.sourceKey}: 항목 매핑 실패: ${error instanceof Error ? error.message : String(error)}`);
242             }
243         }
244     }
245     return collected;
246 }
247
248 async listExternalIds(): Promise<Set<string>> {
249     if (this.useFixtures) {
250         const idsPath = resolve(FIXTURES_DIR, `${this.sourceKey}.ids.json`);
251         try {

```

```

252     const payload = await this.loadJson(idsPath);
253     if (!isRecord(payload) || !Array.isArray(payload.ids)) {
254         throw new CollectorError(`${this.sourceKey}: 잘못된 ID 픽스처`);
255     }
256     return new Set(payload.ids.map((id) => this.externalId(String(id))));
257 } catch (error) {
258     if (!(error instanceof CollectorError) || !error.cause || !isMissingFile(error.cause)) throw
error;
259 }
260 }
261 return new Set((await this.fetch()).map((p) => p.external_id));
262 }
263 }
264
265 export function isRecord(value: unknown): value is Record<string, unknown> {
266     return typeof value === "object" && value !== null && !Array.isArray(value);
267 }
268
269 export function recordOrUndefined(value: unknown): Record<string, unknown> | undefined {
270     return isRecord(value) ? value : undefined;
271 }
272
273 function isMissingFile(error: unknown): boolean {
274     return isRecord(error) && error.code === "ENOENT";
275 }
276
277 export function firstOf(
278     item: Record<string, unknown>,
279     keys: readonly string[],
280     fallback = "",
281 ): string {
282     for (const key of keys) {
283         const value = item[key];
284         if (value !== null && value !== undefined && value !== "") return String(value).trim();
285     }
286     return fallback;
287 }
288
289 export function parseAmount(text: string): [number | null, number | null] {
290     if (!text) return [null, null];
291     const units: Record<string, number> = {
292         억: 100_000_000,
293         천만: 10_000_000,
294         백만: 1_000_000,
295         만: 10_000,
296     };
297     const values = [...text.matchAll(/(\d[\d,]*)\s*(억|천만|백만|만)?\s*(원/g)].map(
298         (match) => Number.parseFloat(match[1].replaceAll(",", "")) * (units[match[2] ?? ""] ?? 1),
299     );
300     if (values.length === 0) return [null, null];
301     if (values.length === 1) {
302         const value = Math.trunc(values[0]);
303         if (/최대|이내|한도|까지/.test(text)) return [null, value];
304         if (/최소|이상|부터/.test(text)) return [value, null];
305         return [value, value];
306     }
307     return [Math.trunc(Math.min(...values)), Math.trunc(Math.max(...values))];
308 }

```

4.2 한국사회보장정보원 복지서비스 수집기

목록조회 (NationalWelfarelistV001) 뒤 신규 항목마다 상세조회 (NationalWelfareDetailedV001)를 호출해 상세 자격 텍스트를 내부 모델로 변환합니다. 목록 API에는 날짜 필터가 없어 `--since` 를 요청에 보내지 않으며, 상세 호출은 일일 예산 안에서 끊습니다. 응답 필드명이 케이스별로 다를 수 있어 후보 키를 순서대로 시도합니다(`firstOf`).

```
1 import { CollectedProgram } from "../models";
2 import {
3     Collector,
4     CollectorError,
5     firstOf,
6     isRecord,
7     PAGINATION_SAFETY_LIMIT,
8     parseAmount,
9     recordOrUndefined,
10    type CollectorOptions,
11    type FetchOptions,
12 } from "../base";
13
14 const LIST_FIELDS = [
15     "inqNum",
16     "intrsThemaArray",
17     "jurMnofNm",
18     "jurOrgNm",
19     "lifeArray",
20     "onapPsblyYn",
21     "rprsCtadr",
22     "servDgst",
23     "servDtlLink",
24     "servId",
25     "servNm",
26     "sprtCycNm",
27     "srvPvsnNm",
28     "trgterIndvdlArray",
29 ] as const;
30
31 const DETAIL_FIELDS = [
32     "alwServCn",
33     "crtYr",
34     "jurMnofNm",
35     "rprsCtadr",
36     "servId",
37     "servNm",
38     "slctCritCn",
39     "tgtrDtlCn",
40     "wlfareInfoOutlCn",
41 ] as const;
42
43 function decodeXml(value: string): string {
44     const named: Record<string, string> = {
45         amp: "&",
46         apos: "'",
47         gt: ">",
48         lt: "<",
49         quot: '"',
50     };
51     return value
52         .replace(/&#(?:x(?:[\da-f]+)|(\d+));/giu, (entity, hex: string, decimal: string) => {
53             const codePoint = Number.parseInt(hex || decimal, hex ? 16 : 10);
54             return codePoint >= 0 && codePoint <= 0x10ffff ? String.fromCodePoint(codePoint) : entity;
55         })
56         .replace(/&(amp|apos|gt|lt|quot);/gu, (_, name: string) => named[name])
57         .replace(/\\r\\n?/gu, "\\n")
58         .trim();
59 }
60
61 function xmlText(xml: string, tag: string): string {
62     return decodeXml(new RegExp(`<${tag}>([\\s\\S]*)</${tag}>`, "u").exec(xml)?.[1] ?? "");
63 }
64
```

```

65 function xmlRecord(xml: string, fields: readonly string[]): Record<string, unknown> {
66     return Object.fromEntries(fields.map((field) => [field, xmlText(xml, field)]));
67 }
68
69 function xmlRecords(xml: string, tag: string, fields: readonly string[]): Record<string, unknown>[] {
70     return [...xml.matchAll(new RegExp(`<${tag}>([\\s\\S]*)</${tag}>`, "gu"))].map(
71         (match) => xmlRecord(match[1], fields),
72     );
73 }
74
75 function requireXmlSuccess(xml: string, sourceKey: string): void {
76     const code = xmlText(xml, "resultCode");
77     if (code !== "0") {
78         throw new CollectorError(`${sourceKey}: ${xmlText(xml, "resultMessage")} || "잘못된 XML 응답"`, {
79             code,
80         });
81     }
82 }
83
84 function ymd(value: string): string | null {
85     const digits = [...value].filter((character) => /\d/.test(character)).join("");
86     return digits.length === 8
87         ? `${digits.slice(0, 4)}-${digits.slice(4, 6)}-${digits.slice(6, 8)}`
88         : null;
89 }
90
91 export class SocialSecurityCollector extends Collector {
92     readonly sourceKey = "social_security";
93     readonly endpoint =
94         "https://apis.data.go.kr/B554287/NationalWelfareInformationsV001/NationalWelfarelistV001";
95     readonly detailEndpoint =
96         "https://apis.data.go.kr/B554287/NationalWelfareInformationsV001/NationalWelfareDetailedV001";
97     readonly idListEndpoint = this.endpoint;
98     override readonly defaultForm = "subsidy";
99     override readonly incrementalStrategy = "full_list_known_ids_detail_budget";
100
101     /**
102      * 상세조회 개발계정 한도는 100회/일 (SPEC §3.2). 목록 호출분과 재시도 여유를
103      * 남기고 그 아래에서 끊는다. 초과하면 429 가 나면서 그 회차 전체가 버려진다.
104      */
105     readonly maxDetailCalls: number;
106
107     constructor(options: CollectorOptions = {}) {
108         super({ pageSize: 500, ...options });
109         this.maxDetailCalls = options.maxDetailCalls ?? 90;
110     }
111
112     protected queryParams({ page }: { since: string | null; page: number }) {
113         return {
114             serviceKey: this.requireApiKey("DATA_GO_KR_API_KEY", this.settings.data_go_kr_api_key),
115             callTp: "L",
116             pageNo: page,
117             numOfRows: this.pageSize,
118             srchKeyCode: "001",
119         };
120     }
121
122     protected items(payload: unknown): Record<string, unknown>[] {
123         if (typeof payload === "string") return xmlRecords(payload, "servList", LIST_FIELDS);
124         if (!isRecord(payload)) return [];
125         const response = recordOrUndefined(payload.response);
126         const body = recordOrUndefined(response?.body) ?? recordOrUndefined(payload.body);
127         let items: unknown = body?.items ?? {};
128         if (isRecord(items)) items = items.item ?? [];
129         if (isRecord(items)) items = [items];
130         return Array.isArray(items) ? items.filter(isRecord) : [];
131     }

```

```

132
133 protected mapItem(item: Record<string, unknown>): CollectedProgram | null {
134     const nativeId = firstOf(item, ["servId", "servid", "SERV_ID"]);
135     const title = firstOf(item, ["servNm", "servnm", "SERV_NM"]);
136     if (!nativeId || !title) return null;
137
138     const outline = firstOf(item, ["wlfareInfoOutlCn", "servDgst", "servDttlLink"]);
139     const target = firstOf(item, ["tgtrDtlCn", "sprtTrgtCn"]);
140     const criteria = firstOf(item, ["slctCritCn"]);
141     const benefit = firstOf(item, ["alwServCn"]);
142     const eligibility = [target, criteria].filter(Boolean).join("\n");
143     const bodyText = [
144         outline,
145         benefit && `[지원내용]\n${benefit}`,
146         eligibility && `[지원대상]\n${eligibility}`,
147     ]
148         .filter(Boolean)
149         .join("\n\n");
150
151     const issuer = firstOf(item, ["jurMnofNm", "jurOrgNm"], "정부");
152     const issuerLevel = ["부", "청", "처", "위원회", "공단"].some((key) => issuer.includes(key))
153         ? "central"
154         : ["특별시", "광역시", "도", "특별자치시", "특별자치도"].some((key) =>
155             issuer.endsWith(key),
156         )
157         ? "metro"
158         : "local";
159     const amountText = firstOf(item, ["sprtAmtCn", "alwServCn"]);
160     const [amountMin, amountMax] = parseAmount(amountText);
161     const startsAt = ymd(firstOf(item, ["enfcBgngYmd"]));
162     const endsAt = ymd(firstOf(item, ["enfcEndYmd"]));
163
164     return new CollectedProgram({
165         external_id: this.externalId(nativeId),
166         source_key: this.sourceKey,
167         source_url: firstOf(
168             item,
169             ["servDttlLink", "servUrl"],
170             `https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=${nativeId}`,
171         ),
172         title,
173         body_text: bodyText || title,
174         eligibility_text: eligibility,
175         form: this.defaultForm,
176         issuer,
177         issuer_level: issuerLevel,
178         benefit_amount_text: amountText,
179         benefit_amount_min: amountMin,
180         benefit_amount_max: amountMax,
181         apply_url: firstOf(item, ["applmetUrl"]),
182         apply_method: firstOf(item, ["applmetCn", "srvPvsnNm"]),
183         starts_at: startsAt,
184         ends_at: endsAt,
185         is_always_open: endsAt === null,
186         raw_body: item,
187     });
188 }
189
190 private async listPage(
191     page: number,
192 ): Promise<{ items: Record<string, unknown>[]; totalCount: number | null }> {
193     const payload = await this.getXml(this.queryParams({ since: null, page }));
194     requireXmlSuccess(payload, this.sourceKey);
195     const totalCountText = xmlText(payload, "totalCount");
196     const totalCount = totalCountText ? Number(totalCountText) : null;
197     return {

```

```

198         items: this.items(payload),
199         totalCount:
200             totalCount !== null && Number.isInteger(totalCount) && totalCount >= 0 ? totalCount : null,
201     };
202 }
203
204 private async getXml(
205     params: Record<string, string | number>,
206     endpoint = this.endpoint,
207 ): Promise<string> {
208     return this.request(
209         params,
210         async (response) => {
211             const xml = await response.text();
212             if (!response.headers.get("content-type")?.includes("xml") && !xml.trimStart().startsWith("<")) {
213                 throw new CollectorError(`${this.sourceKey}: 잘못된 XML 응답 형식`);
214             }
215             return xml;
216         },
217         endpoint,
218     );
219 }
220
221 private async detail(nativeId: string): Promise<Record<string, unknown>> {
222     const payload = await this.getXml(
223         {
224             serviceKey: this.requireApiKey("DATA_GO_KR_API_KEY", this.settings.data_go_kr_api_key),
225             callTp: "D",
226             servId: nativeId,
227         },
228         this.detailEndpoint,
229     );
230     requireXmlSuccess(payload, this.sourceKey);
231     const item = xmlRecord(payload, DETAIL_FIELDS);
232     item.applmetCn = xmlRecords(payload, "applmetList", ["servSeDetailLink"])
233         .map((entry) => String(entry.servSeDetailLink || ""))
234         .filter(Boolean)
235         .join("\n");
236     return item;
237 }
238
239 /**
240  * 건당 상세 조회가 필요한데 개발계정 한도가 100회/일이라, 전량(수백 건)을 한 번에
241  * 받을 수 없다. 세 가지로 나눠 처리한다.
242  * 1. 이미 적재된 건은 상세를 건너뛰다 - 한도를 신규 건에 몰아준다
243  * 2. 한 회차의 상세 호출을 maxDetailCalls 로 끊는다
244  * 3. 그래도 실패하면 던지지 않고 여기까지 모은 것을 반환한다
245  * 3이 핵심이다. 예전에는 중간에 429가 나면 CollectorError 로 그 회차 전체가
246  * 버려져, 매일 처음부터 다시 시작하며 초기 적재가 영영 끝나지 않았다.
247  *
248  * ponytail: 적재된 건은 다시 안 보므로 원본 수정(마감 연장·자격 완화)을 놓친다.
249  * 한도가 풀리면(운영계정) knownIds 스킵을 빼고 content_hash 비교로 되돌린다.
250  */
251 override async fetch(options: FetchOptions = {}): Promise<CollectedProgram[]> {
252     const { since, knownIds } = options;
253     const maxPages = options.maxPages ?? PAGINATION_SAFETY_LIMIT;
254     // 이 공식 목록 API에는 날짜 필터가 없다. --since 는 요청 파라미터로 보내지 않고,
255     // 전체 목록을 훑은 뒤 knownIds 로 기존 건을 건너뛰고 상세조회 예산을 신규 건에 쓴다.
256     void since;
257     if (this.useFixtures) return super.fetch({ maxPages });
258     this.observedCount = 0;
259     this.errors.length = 0;
260
261     const collected: CollectedProgram[] = [];
262     const seen = new Set<string>();
263     let budget = this.maxDetailCalls;
264     let skipped = 0;

```

```

265 let complete = false;
266 for (let page = 1; page <= maxPages; page++) {
267     const { items, totalCount } = await this.listPage(page);
268     this.observCount = totalCount ?? (this.observCount ?? 0) + items.length;
269     for (const item of items) {
270         const nativeId = firstOf(item, ["servId"]);
271         if (!nativeId || seen.has(nativeId)) continue;
272         if (knownIds?.has(this.externalId(nativeId))) continue;
273         // 포털은 논리적 호출이 아니라 HTTP 요청 수로 한도를 센다. detail() 한 번이
274         // 재시도까지 최대 retries+1 회 나갈 수 있으므로, 그만큼 남아 있을 때만
275         // 시작하고 실제 나간 요청 수만큼 깎는다. 남은 예산만 보고 시작하면
276         // 마지막 한 건이 재시도로 한도를 넘겨버린다.
277         if (budget < this.retries + 1) {
278             skipped++;
279             continue;
280         }
281
282         const httpBefore = this.httpCalls;
283         let detail: Record<string, unknown>;
284         try {
285             detail = await this.detail(nativeId);
286             budget -= this.httpCalls - httpBefore;
287         } catch (error) {
288             budget -= this.httpCalls - httpBefore;
289             const message = error instanceof Error ? error.message : String(error);
290             const missingDetail =
291                 error instanceof CollectorError &&
292                 (error.status === 404 ||
293                 error.status === 410 ||
294                 (error.code !== undefined && Number(error.code) === 3));
295             if (missingDetail) {
296                 seen.add(nativeId);
297                 console.warn(`${this.sourceKey}: 상세 조회 건너뛴 (${nativeId}: ${message})`);
298                 continue;
299             }
300             console.warn(
301                 `${this.sourceKey}: 상세 조회 중단 (${message}) - ${collected.length}건까지 저장하고 다음 회차에
이어받습니다`,
302             );
303             this.errors.push(`${this.sourceKey}: 상세 조회 중단: ${message}`);
304             return collected;
305         }
306
307         const program = this.mapItem({ ...item, ...detail });
308         if (!program) continue;
309         seen.add(nativeId);
310         collected.push(program);
311     }
312     if (items.length < this.pageSize || (totalCount !== null && page * this.pageSize >= totalCount)) {
313         complete = true;
314         break;
315     }
316 }
317 if (!complete && options.maxPages === undefined) {
318     throw new CollectorError(`${this.sourceKey}: 페이지 안전 한도에 도달해 전량 목록 여부를 확인할 수 없음`);
319 }
320 if (skipped) {
321     console.warn(
322         `${this.sourceKey}: 일일 상세 호출 한도(${this.maxDetailCalls})로 ${skipped}건을 다음 회차로 미룹니다`,
323     );
324 }
325 return collected;
326 }
327
328 override async listExternalIds(): Promise<Set<string>> {
329     if (this.useFixtures) return super.listExternalIds();
330     const ids = new Set<string>();

```



```

331     let complete = false;
332     for (let page = 1; page <= PAGINATION_SAFETY_LIMIT; page++) {
333         const { items, totalCount } = await this.listPage(page);
334         for (const item of items) {
335             const nativeId = firstOf(item, ["servId"]);
336             if (nativeId) ids.add(this.externalId(nativeId));
337         }
338         if (items.length < this.pageSize || (totalCount !== null && page * this.pageSize >= totalCount)) {
339             complete = true;
340             break;
341         }
342     }
343     if (!complete) throw new CollectorError(`${this.sourceKey}: ID 전량 대조 페이지 안전 한도 도달`);
344     return ids;
345 }
346 }

```

4.3 지원사업 공고 수집기

web/ingest/collectors/bizinfo.ts

110 lines · TypeScript

```

1  import { CollectedProgram } from "../models";
2  import { Collector, firstOf, isRecord, parseAmount, recordOrUndefined } from "../base";
3
4  function splitPeriod(text: string): [string | null, string | null, boolean] {
5      if (!text) return [null, null, false];
6      const dates = [...text.matchAll(/(\d{4})[./-]?(\d{2})[./-]?(\d{2})/g)].map(
7          (match) => `${match[1]}-${match[2]}-${match[3]}`,
8      );
9      const always = /예산\s*소진|상시|연중/.test(text);
10     if (dates.length >= 2) return [dates[0], dates[1], false];
11     if (dates.length === 1) return [dates[0], null, true];
12     return [null, null, always];
13 }
14
15 export class BizinfoCollector extends Collector {
16     readonly sourceKey = "bizinfo";
17     readonly endpoint = "https://www.bizinfo.go.kr/uss/rss/bizinfoApi.do";
18     readonly idListEndpoint = this.endpoint;
19     override readonly defaultForm = "loan";
20
21     protected queryParams({ since, page }: { since: string | null; page: number }) {
22         const params: Record<string, string | number> = {
23             crtfcKey: this.requireApiKey("BIZINFO_API_KEY", this.settings.bizinfo_api_key),
24             dataType: "json",
25             pageUnit: this.pageSize,
26             pageIndex: page,
27         };
28         if (since) params.searchBgnDe = since.replaceAll("-", "");
29         return params;
30     }
31
32     protected items(payload: unknown): Record<string, unknown>[] {
33         if (!isRecord(payload)) return [];
34         const response = recordOrUndefined(payload.response);
35         const body = recordOrUndefined(response?.body);
36         let items: unknown = payload.jsonArray ?? body?.items;
37         if (isRecord(items)) items = [items];
38         return Array.isArray(items) ? items.filter(isRecord) : [];
39     }
40
41     protected mapItem(item: Record<string, unknown>): CollectedProgram | null {
42         const nativeId = firstOf(item, ["pblancId", "pblance_id"]);
43         const title = firstOf(item, ["pblancNm", "pblanc_nm"]);

```

```

44     if (!nativeId || !title) return null;
45
46     const summary = firstOf(item, ["bsnsSumryCn", "bsns_sumry_cn"]);
47     const target = firstOf(item, ["trgetNm", "trget_nm"]);
48     const realm = firstOf(item, ["pldirSportRealmLclasCodeNm"]);
49     const bodyText = [
50         summary,
51         realm && `[자원분야] ${realm}`,
52         target && `[지원대상]\n${target}`,
53     ]
54     .filter(Boolean)
55     .join("\n\n");
56     const issuer = firstOf(item, ["jrdsInsttNm", "excInsttNm"], "중소벤처기업부");
57     const issuerLevel = [
58         "서울",
59         "부산",
60         "대구",
61         "인천",
62         "광주",
63         "대전",
64         "울산",
65         "세종",
66         "경기",
67         "강원",
68         "충청",
69         "전라",
70         "경상",
71         "제주",
72         "전북",
73         "전남",
74         "경북",
75         "경남",
76         "충북",
77         "충남",
78     ].some((prefix) => issuer.startsWith(prefix))
79         ? "metro"
80         : "central";
81     const benefitText = firstOf(item, ["sportScvlNm", "sportAmtCn"]);
82     const [amountMin, amountMax] = parseAmount(benefitText || summary);
83     const [startsAt, endsAt, always] = splitPeriod(firstOf(item, ["reqstBeginEndDe"]));
84
85     return new CollectedProgram({
86         external_id: this.externalId(nativeId),
87         source_key: this.sourceKey,
88         source_url: firstOf(
89             item,
90             ["pblancUrl", "rceptEngnHmpgUrl"],
91             `https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?pblancId=${nativeId}`,
92         ),
93         title,
94         body_text: bodyText || title,
95         eligibility_text: target,
96         form: this.defaultForm,
97         issuer,
98         issuer_level: issuerLevel,
99         benefit_amount_text: benefitText,
100        benefit_amount_min: amountMin,
101        benefit_amount_max: amountMax,
102        apply_url: firstOf(item, ["rceptEngnHmpgUrl", "pblancUrl"]),
103        apply_method: firstOf(item, ["reqstMthPapersCn", "rceptMthNm"], "온라인 신청"),
104        starts_at: startsAt,
105        ends_at: endsAt,
106        is_always_open: always && endsAt === null,
107        raw_body: item,
108    });
109 }

```

4.4 금융상품 공시 수집기

기본정보(`baseList`)와 옵션정보(`optionList`)를 상품코드로 결합하는 구조를 처리합니다.

web/ingest/collectors/finlife.ts

209 lines · TypeScript

```

1  import { CollectedProgram } from "../models";
2  import {
3    Collector,
4    CollectorError,
5    firstOf,
6    isRecord,
7    PAGINATION_SAFETY_LIMIT,
8    parseAmount,
9    recordOrUndefined,
10   type FetchOptions,
11 } from "./base";
12
13 export const TOP_FIN_GRP_NO = "020000";
14 export const FINLIFE_GROUPS = ["020000", "030200", "030300", "050000", "060000"] as const;
15
16 const PRODUCTS = [
17   ["deposit", "depositProductsSearch", "예금", "product"],
18   ["saving", "savingProductsSearch", "적금", "product"],
19   ["annuity", "annuitySavingProductsSearch", "연금저축", "product"],
20   ["mortgage", "mortgageLoanProductsSearch", "주택담보대출", "loan"],
21   ["rent", "rentHouseLoanProductsSearch", "전세자금대출", "loan"],
22   ["credit", "creditLoanProductsSearch", "개인신용대출", "loan"],
23 ] as const;
24
25 export const FINLIFE_PRODUCTS = PRODUCTS.map(([, endpoint]) => endpoint);
26
27 const API_ROOT = "https://finlife.fss.or.kr/finlifeapi";
28 const SOURCE_URL = "https://finlife.fss.or.kr/finlife/main/contents.do?menuNo=700029";
29 const NO_ELIGIBILITY_INFO = "[자격요건 정보 없음]";
30
31 function formatRate(value: unknown): string {
32   return typeof value === "number" && Number.isInteger(value) ? value.toFixed(1) : String(value ?? "");
33 }
34
35 function optionLine(option: Record<string, unknown>): string {
36   if (option.intr_rate !== undefined || option.intr_rate2 !== undefined) {
37     return `${String(option.save_trm ?? "")}개월 ${String(option.intr_rate_type_nm ?? "")} 기본
38     ${formatRate(option.intr_rate)}% / 최고 ${formatRate(option.intr_rate2)}%\`.trim();
39   }
40   const labels: [string, string][] = [
41     ["mrtg_type_nm", "담보유형"],
42     ["rpay_type_nm", "상환방식"],
43     ["lend_rate_type_nm", "대출금리유형"],
44     ["lend_rate_min", "최저금리(%)"],
45     ["lend_rate_max", "최고금리(%)"],
46     ["lend_rate_avg", "평균금리(%)"],
47     ["crdt_grad_avg", "평균금리(%)"],
48     ["dccls_rate", "공시이율(%)"],
49     ["guar_rate", "최저보증이율(%)"],
50 ];
51   return labels
52     .flatMap(([key, label]) =>
53       option[key] === null || option[key] === undefined || option[key] === ""
54       ? []
55       : [` ${label} ${formatRate(option[key])}`],
56     )

```

```

56     .join(" · ");
57 }
58
59 export class FinlifeCollector extends Collector {
60     readonly sourceKey = "finlife";
61     readonly endpoint = `${API_ROOT}/depositProductsSearch.json`;
62     readonly idListEndpoint = this.endpoint;
63     override readonly defaultForm = "product";
64
65     protected queryParams({ page }: { since: string | null; page: number }) {
66         return {
67             auth: this.requireApiKey("FINLIFE_API_KEY", this.settings.finlife_api_key),
68             topFinGrpNo: TOP_FIN_GRP_NO,
69             pageNo: page,
70         };
71     }
72
73     protected items(payload: unknown): Record<string, unknown>[] {
74         if (!isRecord(payload)) return [];
75         const result = recordOrUndefined(payload.result) ?? {};
76         const base = Array.isArray(result.baseList) ? result.baseList : [];
77         const options = Array.isArray(result.optionList) ? result.optionList : [];
78         const byProduct = new Map<string, Record<string, unknown>[]>();
79         for (const option of options) {
80             if (!isRecord(option)) continue;
81             const code = String(option.fin_prdt_cd);
82             byProduct.set(code, [...(byProduct.get(code) ?? []), option]);
83         }
84         return base.filter(isRecord).map((row) => ({
85             ...row,
86             _options: byProduct.get(String(row.fin_prdt_cd)) ?? [],
87         }));
88     }
89
90     override async fetch(options: FetchOptions = {}): Promise<CollectedProgram[]> {
91         if (this.useFixtures) return super.fetch(options);
92         this.observedCount = 0;
93         this.errors.length = 0;
94         const maxPages = options.maxPages ?? PAGINATION_SAFETY_LIMIT;
95         const collected: CollectedProgram[] = [];
96         const seen = new Set<string>();
97         const auth = this.requireApiKey("FINLIFE_API_KEY", this.settings.finlife_api_key);
98
99         for (const [kind, endpointName, label, form] of PRODUCTS) {
100             for (const group of FINLIFE_GROUPS) {
101                 let complete = false;
102                 for (let page = 1; page <= maxPages; page++) {
103                     const payload = await this.request(
104                         { auth, topFinGrpNo: group, pageNo: page },
105                         async (response) => (await response.json()) as unknown,
106                         `${API_ROOT}/${endpointName}.json`,
107                     );
108                     const result = isRecord(payload) ? recordOrUndefined(payload.result) : undefined;
109                     if (!result || String(result.err_cd) !== "000" || !Array.isArray(result.baseList)) {
110                         throw new CollectorError(
111                             `${this.sourceKey}: ${endpointName}/${group} 오류 또는 잘못된 응답 봉투${result?.err_msg ? `: ${String(result.err_msg)}` : ""}`,
112                         );
113                     }
114                     const rows = this.items(payload).map((row) => ({
115                         ...row,
116                         _productKind: kind,
117                         _productLabel: label,
118                         _productForm: form,
119                         _topFinGrpNo: group,
120                     }));
121                     this.observedCount = (this.observedCount ?? 0) + rows.length;

```

```

122     for (const row of rows) {
123         const program = this.mapItem(row);
124         if (!program || seen.has(program.external_id)) continue;
125         seen.add(program.external_id);
126         collected.push(program);
127     }
128
129     const hasLastPage = result.max_page_no !== undefined &&
130         result.max_page_no !== null && result.max_page_no !== "";
131     const lastPage = Number(result.max_page_no);
132     if ((hasLastPage && Number.isInteger(lastPage) && lastPage >= 0)
133         ? page >= lastPage
134         : rows.length < this.pageSize) {
135         complete = true;
136         break;
137     }
138 }
139 if (!complete && options.maxPages === undefined) {
140     throw new CollectorError(`${this.sourceKey}: ${endpointName}/${group} 페이지 안전 한도 도달`);
141 }
142 }
143 }
144 return collected;
145 }
146
147 protected mapItem(item: Record<string, unknown>): CollectedProgram | null {
148     const productCode = firstOf(item, ["fin_prdt_cd"]);
149     const companyCode = firstOf(item, ["fin_co_no"]);
150     const productName = firstOf(item, ["fin_prdt_nm"]);
151     if (!productCode || !productName) return null;
152
153     const kind = firstOf(item, ["_productKind"], "deposit");
154     const productLabel = firstOf(item, ["_productLabel"], "예금");
155     const group = firstOf(item, ["_topFinGrpNo"], TOP_FIN_GRP_NO);
156     const company = firstOf(item, ["kor_co_nm"], "금융회사");
157     const joinWay = firstOf(item, ["join_way"]);
158     const joinMember = firstOf(item, ["join_member"]);
159     const special = firstOf(item, ["spcl_cnd"]);
160     const note = firstOf(item, ["etc_note"]);
161     const deny = firstOf(item, ["join_deny"]);
162     const options = Array.isArray(item._options) ? item._options.filter(isRecord) : [];
163     const optionLines = options.map(optionLine).filter(Boolean).map((line) => `- ${line}`);
164     const bodyText = [
165         `${company}의 ${productLabel} 상품 ${productName}입니다.`,
166         joinWay && `[가입방법] ${joinWay}`,
167         optionLines.length && `[금리·조건]\n${optionLines.join("\n")}`,
168         special && `[우대조건]\n${special}`,
169         note && `[유의사항]\n${note}`,
170         joinMember && `[가입대상]\n${joinMember}`,
171         deny && `[가입제한]\n${deny}`,
172         firstOf(item, ["loan_inci_expn"]) && `[부대비용] ${firstOf(item, ["loan_inci_expn"])}`,
173         firstOf(item, ["erly_rpay_fee"]) && `[종도상환수수료] ${firstOf(item, ["erly_rpay_fee"])}`,
174         firstOf(item, ["dly_rate"]) && `[연체이율] ${firstOf(item, ["dly_rate"])}`,
175     ]
176     .filter(Boolean)
177     .join("\n\n");
178     const limitText = firstOf(item, ["max_limit", "loan_lmt"]);
179     let [amountMin, amountMax] = parseAmount(/^\d+$/ .test(limitText) ? `${limitText}원` : limitText);
180     if (/^\d+$/ .test(limitText)) {
181         amountMin = null;
182         amountMax = Number.parseInt(limitText, 10);
183     }
184     const nativeId = kind === "deposit"
185         ? `${companyCode}-${productCode}`
186         : `${kind}-${group}-${companyCode}-${productCode}`;
187
188     return new CollectedProgram({

```

```

189     external_id: this.externalId(nativeId),
190     source_key: this.sourceKey,
191     source_url: SOURCE_URL,
192     title: `[${company}] ${productName}`,
193     body_text: bodyText,
194     eligibility_text: joinMember || NO_ELIGIBILITY_INFO,
195     form: firstOf(item, ["_productForm"], this.defaultForm) === "loan" ? "loan" : "product",
196     issuer: company,
197     issuer_level: "central",
198     benefit_amount_text: limitText ? `최고한도 ${limitText}${/^\\d+$/.test(limitText) ? "원" : ""}` : "",
199     benefit_amount_min: amountMin,
200     benefit_amount_max: amountMax,
201     apply_url: "https://finlife.fss.or.kr/",
202     apply_method: joinWay || "금융회사 확인",
203     starts_at: null,
204     ends_at: null,
205     is_always_open: true,
206     raw_body: item,
207   });
208 }
209 }

```

4.5 수집기 등록

web/ingest/collectors/index.ts

33 lines · TypeScript

```

1  import { BizinfoCollector } from "../bizinfo";
2  import type { CollectorConstructor } from "../base";
3  import { FinlifeCollector } from "../finlife";
4  import { Gov24Collector } from "../gov24";
5  import { KstartupCollector } from "../kstartup";
6  import { LocalWelfareCollector } from "../local-welfare";
7  import { SocialSecurityCollector } from "../social-security";
8
9  export { BizinfoCollector } from "../bizinfo";
10 export { Collector, CollectorError, firstOf, parseAmount } from "../base";
11 export type { CollectorConstructor, CollectorOptions, FetchLike, FetchOptions } from "../base";
12 export { FinlifeCollector, TOP_FIN_GRP_NO } from "../finlife";
13 export { Gov24Collector } from "../gov24";
14 export { KstartupCollector } from "../kstartup";
15 export { LocalWelfareCollector } from "../local-welfare";
16 export { SocialSecurityCollector } from "../social-security";
17
18 export const COLLECTORS: Record<string, CollectorConstructor> = {
19   bizinfo: BizinfoCollector,
20   finlife: FinlifeCollector,
21   gov24: Gov24Collector,
22   kstartup: KstartupCollector,
23   local_welfare: LocalWelfareCollector,
24   social_security: SocialSecurityCollector,
25 };
26
27 export const DEFAULT_SOURCE_KEYS = [
28   "bizinfo",
29   "finlife",
30   "gov24",
31   "local_welfare",
32   "social_security",
33 ] as const;

```

4.6 설정 — 인증키 주입

인증키는 **소스코드에 포함하지 않고** 환경변수로만 주입합니다. 저장소에는 키가 커밋되지 않으며, 실행 환경(GitHub Actions Secrets)에서 공급합니다. 공공데이터포털 경우 소스는 `DATA_GO_KR_API_KEY` 하나를 공유하고, 자체 창구에서 키를 발급하는 소스는 각자의 환경변수(`BIZINFO_API_KEY` , `FINLIFE_API_KEY`)를 사용하여 **포털 인증키가 포털 외부로 전송되지 않도록** 분리했습니다.

web/ingest/config.ts

41 lines · TypeScript

```
1  import { dirname, resolve } from "node:path";
2  import { fileURLToPath } from "node:url";
3
4  export const PACKAGE_DIR = dirname(fileURLToPath(import.meta.url));
5  export const REPO_ROOT = resolve(PACKAGE_DIR, "../..");
6  export const SHARED_DIR = resolve(REPO_ROOT, "shared");
7  export const FIXTURES_DIR = resolve(REPO_ROOT, "ingest/fixtures");
8
9  export const EMBEDDING_DIM = 1024;
10
11 export interface Settings {
12   database_url: string;
13   data_go_kr_api_key: string;
14   bizinfo_api_key: string;
15   finlife_api_key: string;
16   embedding_provider: string;
17   embedding_api_key: string;
18   mock_embeddings: boolean;
19 }
20
21 function envValue(env: NodeJS.ProcessEnv, name: string, fallback = ""): string {
22   return (env[name] || fallback).trim();
23 }
24
25 export function settingsFromEnv(env: NodeJS.ProcessEnv = process.env): Settings {
26   const provider = envValue(env, "EMBEDDING_PROVIDER", "mock").toLowerCase() || "mock";
27   const forcedMock = envValue(env, "MOCK_EMBEDDINGS") === "1";
28   const embeddingKey = envValue(env, "EMBEDDING_API_KEY");
29
30   return {
31     database_url: envValue(env, "DATABASE_URL"),
32     data_go_kr_api_key: envValue(env, "DATA_GO_KR_API_KEY"),
33     bizinfo_api_key: envValue(env, "BIZINFO_API_KEY"),
34     finlife_api_key: envValue(env, "FINLIFE_API_KEY"),
35     embedding_provider: forcedMock ? "mock" : provider,
36     embedding_api_key: embeddingKey,
37     mock_embeddings: forcedMock || provider === "mock" || !embeddingKey,
38   };
39 }
40
41 export const loadSettings = settingsFromEnv;
```

5. API 응답 처리 계약 (JSON)

키 발급 전 개발·테스트를 위해 **API 응답 봉투 구조를 그대로 모사한 픽스처**를 사용했습니다. 실제 API를 호출하지 않고도 파싱·저장 로직을 검증하기 위한 것으로, 키 발급 후에는 실제 응답으로 필드 명세를 확정합니다.

5.1 복지서비스 응답 구조

ingest/fixtures/social_security.json

232 lines · JSON

```
1  {
2    "_fixture_note": "한국사회보장정보원 사회보장급여 목록 API 응답 '봉투 그대로' 흉내낸 픽스처. 필드명·구조는 W1 실물 검증 대상 (SPEC §3.3). inqNum(조회수)과 body.resultTime 은 매 호출 바뀌는 휘발성 필드로, content_hash 계산에서 제외되는지 확인하는 용도로 일부러 넣었다 (SPEC §3.2).",
3    "response": {
4      "header": { "resultCode": "00", "resultMsg": "NORMAL SERVICE." },
5      "body": {
6        "numOfRows": 12,
7        "pageNo": 1,
8        "totalCount": 12,
9        "resultTime": "2026-08-08T18:12:03+09:00",
10       "items": {
11         "item": [
12           {
13             "servId": "WII0000345",
14             "servNm": "청년월세 한시 특별지원",
15             "jurMnofNm": "국토교통부",
16             "jurOrgNm": "국토교통부",
17             "wlfareInfoOutlCn": "부모와 따로 거주하는 저소득 청년의 주거비 부담을 덜기 위해 월세를 한시적으로 지원하는 사업입니다.",
18             "sprtTrgtCn": "만 19세 이상 34세 이하 무주택 청년으로 부모와 별도 거주하는 자",
19             "slctCritCn": "청년 본인 가구 기준 중위소득 60% 이하이며, 원가구 기준 중위소득 100% 이하인 경우 선정합니다. 임차보증금 5천만원 이하 및 월세 60만원 이하 주택 거주자에 한합니다.",
20             "alwServCn": "월 최대 20만원을 12개월간 지원",
21             "sprtAmtCn": "월 최대 20만원",
22             "applmetCn": "복지로 온라인 신청 또는 주소지 행정복지센터 방문 신청",
23             "applmetUrl": "https://www.bokjiro.go.kr/",
24             "servDtllink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII0000345",
25             "enfcBgngYmd": "20260301",
26             "enfcEndYmd": "20261231",
27             "srvPvsnm": "현금",
28             "inqNum": "184203"
29           },
30           {
31             "servId": "WII0000112",
32             "servNm": "주거급여(임차급여)",
33             "jurMnofNm": "국토교통부",
34             "jurOrgNm": "국토교통부",
35             "wlfareInfoOutlCn": "소득과 주거 형태를 고려해 임차료를 지원하여 주거 안정을 돕는 제도입니다.",
36             "sprtTrgtCn": "임차료를 부담하는 임차가구",
37             "slctCritCn": "기준 중위소득 48% 이하 가구를 대상으로 하며, 부양의무자 기준은 적용하지 않습니다. 실제 임차료를 지불하고 있어야 합니다.",
38             "alwServCn": "지역·가구원수별 기준임대로 범위에서 실제 임차료 지급",
39             "sprtAmtCn": "월 최대 63만원",
40             "applmetCn": "주민등록 주소지 행정복지센터 방문 신청",
41             "applmetUrl": "https://www.bokjiro.go.kr/",
42             "servDtllink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII0000112",
43             "enfcBgngYmd": "20260101",
44             "enfcEndYmd": "",
45             "srvPvsnm": "현금",
46             "inqNum": "902114"
```



```

    },
    {
        "servId": "WII0000001",
        "servNm": "기초연금",
        "jurMnofNm": "보건복지부",
        "jurOrgNm": "보건복지부",
        "wlfareInfoOutlCn": "어르신들의 안정된 노후생활을 돕기 위해 매월 연금을 지급하는 제도입니다.",
        "sprtTrgtCn": "만 65세 이상 어르신 중 소득인정액이 선정기준액 이하인 분",
        "slctCritCn": "소득 하위 70%에 해당하는 어르신을 선정합니다. 공무원연금 등 직역연금 수급자는 원칙적으로 제외됩니다.",
        "alwServCn": "월 최대 34만원 지급",
        "sprtAmtCn": "월 최대 34만원",
        "applmetCn": "국민연금공단 지사 또는 행정복지센터 방문 신청, 복지로 온라인 신청",
        "applmetUrl": "https://www.bokjiro.go.kr/",
        "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII0000001",
        "enfcBgngYmd": "20260101",
        "enfcEndYmd": "",
        "srvPvsnNm": "현금",
        "inqNum": "1520932"
    },
    {
        "servId": "WII00000078",
        "servNm": "긴급복지지원(생계지원)",
        "jurMnofNm": "보건복지부",
        "jurOrgNm": "보건복지부",
        "wlfareInfoOutlCn": "갑작스러운 위기상황으로 생계유지가 곤란한 가구에 생계비를 신속하게 지원합니다.",
        "sprtTrgtCn": "주소득자의 사망·질병·실직 등으로 소득을 상실한 위기가구",
        "slctCritCn": "기준 중위소득 75% 이하이면서 재산 및 금융재산 기준을 충족해야 합니다. 동일 사유로 중복 지원 불가합니다.",
        "alwServCn": "4인 가구 기준 월 187만원 이내 생계비 지급",
        "sprtAmtCn": "월 최대 187만원",
        "applmetCn": "시군구청 또는 보건복지상담센터(129) 신청",
        "applmetUrl": "https://www.bokjiro.go.kr/",
        "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII00000078",
        "enfcBgngYmd": "20260101",
        "enfcEndYmd": "",
        "srvPvsnNm": "현금",
        "inqNum": "331002"
    },
    {
        "servId": "WII00000254",
        "servNm": "노인맞춤돌봄서비스",
        "jurMnofNm": "보건복지부",
        "jurOrgNm": "보건복지부",
        "wlfareInfoOutlCn": "일상생활 영위가 어려운 취약 어르신에게 안전지원, 사회참여, 생활교육 등 맞춤형 돌봄을 제공합니다.",
        "sprtTrgtCn": "만 65세 이상 국민기초생활수급자, 차상위계층 또는 기초연금 수급자",
        "slctCritCn": "독거·조손 등 돌봄이 필요한 어르신을 우선 선정합니다. 장기요양 등급판정자는 제외합니다.",
        "alwServCn": "안전지원·사회참여·생활교육 등 서비스 제공",
        "sprtAmtCn": "",
        "applmetCn": "주소지 행정복지센터 방문 신청",
        "applmetUrl": "https://www.bokjiro.go.kr/",
        "servDtLlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?wlfareInfoId=WII00000254",
        "enfcBgngYmd": "20260101",
        "enfcEndYmd": "",
        "srvPvsnNm": "서비스",
        "inqNum": "88231"
    },
    {
        "servId": "WII00000411",
        "servNm": "에너지바우처",
        "jurMnofNm": "산업통상자원부",
        "jurOrgNm": "한국에너지공단",
        "wlfareInfoOutlCn": "냉난방 에너지 구입비를 지원해 취약계층의 계절 부담을 줄이는 사업입니다.",

```

```

108         "sprtTrgtCn": "국민기초생활수급 가구 중 노인·영유아·장애인·임산부 등이 포함된 세대",
109         "slctCritCn": "생계급여 또는 의료급여 수급 가구여야 하며, 세대원 중 더위·추위 민감계층이 있어야 합니다.",
110         "alwServCn": "가구원 수에 따라 연 최대 70만원 상당 바우처 지급",
111         "sprtAmtCn": "연 최대 70만원",
112         "applmetCn": "행정복지센터 방문 신청 또는 복지로 온라인 신청",
113         "applmetUrl": "https://www.energyv.or.kr/",
114         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000411",
115         "enfcBgngYmd": "20260601",
116         "enfcEndYmd": "20270430",
117         "srvPvsnNm": "이용권",
118         "inqNum": "45120"
119     },
120     {
121         "servId": "WII0000633",
122         "servNm": "농어촌 주거환경 개선 융자",
123         "jurMnofNm": "농림축산식품부",
124         "jurOrgNm": "전라남도 순천시",
125         "wlfareInfoOutlCn": "노후 농어촌 주택의 개량과 신축에 필요한 자금을 장기 저리로 융자합니다.",
126         "sprtTrgtCn": "전라남도 순천시에 거주하는 농업인 또는 어업인 세대주",
127         "slctCritCn": "해당 지역에 1년 이상 계속 거주하여야 하며, 주택 노후도 심사를 거쳐 선정합니다.",
128         "alwServCn": "가구당 최대 2억원 융자, 연 2% 고정금리",
129         "sprtAmtCn": "최대 2억원",
130         "applmetCn": "순천시청 농업정책과 방문 신청",
131         "applmetUrl": "https://www.suncheon.go.kr/",
132         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000633",
133         "enfcBgngYmd": "20260401",
134         "enfcEndYmd": "20261130",
135         "srvPvsnNm": "융자",
136         "inqNum": "2210"
137     },
138     {
139         "servId": "WII0000702",
140         "servNm": "서울시 청년월세지원",
141         "jurMnofNm": "서울특별시",
142         "jurOrgNm": "서울특별시",
143         "wlfareInfoOutlCn": "서울에 거주하는 청년 1인 가구의 월세를 지원해 주거비 부담을 낮춥니다.",
144         "sprtTrgtCn": "서울특별시에 주민등록을 둔 만 19세 이상 39세 이하 청년 1인 가구",
145         "slctCritCn": "기준 중위소득 150% 이하이며 임차보증금 8천만원 이하, 월세 60만원 이하 주택에 거주해야 합니다.
국토교통부 청년월세 특별지원과 중복 수혜 불가합니다.",
146         "alwServCn": "월 20만원씩 최대 12개월 지원",
147         "sprtAmtCn": "월 20만원",
148         "applmetCn": "서울주거포털 온라인 신청",
149         "applmetUrl": "https://housing.seoul.go.kr/",
150         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000702",
151         "enfcBgngYmd": "20260501",
152         "enfcEndYmd": "20260930",
153         "srvPvsnNm": "현금",
154         "inqNum": "77410"
155     },
156     {
157         "servId": "WII0000188",
158         "servNm": "아이돌봄서비스",
159         "jurMnofNm": "여성가족부",
160         "jurOrgNm": "여성가족부",
161         "wlfareInfoOutlCn": "가정으로 아이돌보미가 찾아가 아동을 안전하게 돌봐주는 서비스입니다.",
162         "sprtTrgtCn": "만 12세 이하 아동을 양육하는 가정",
163         "slctCritCn": "기준 중위소득 150% 이하 가구는 이용요금의 일부를 정부가 지원합니다. 맞벌이·다자녀 가정을 우선
선정합니다.",
164         "alwServCn": "시간제·영아종일제 돌봄 서비스 및 이용요금 차등 지원",
165         "sprtAmtCn": "시간당 최대 12,180원 지원",
166         "applmetCn": "아이돌봄서비스 누리집 온라인 신청",
167         "applmetUrl": "https://www.idolbom.go.kr/",
168         "servDtlLink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000188",

```

```

169         "enfcBgngYmd": "20260101",
170         "enfcEndYmd": "",
171         "srvPvsNm": "서비스",
172         "inqNum": "412009"
173     },
174     {
175         "servId": "WII0000520",
176         "servNm": "청년 마음건강 지원사업",
177         "jurMnofNm": "보건복지부",
178         "jurOrgNm": "보건복지부",
179         "wlfareInfoOutlCn": "정서적 어려움을 겪는 청년에게 전문 심리상담 서비스를 제공합니다.",
180         "sprtTrgtCn": "만 19세 이상 34세 이하 청년",
181         "slctCritCn": "소득 기준은 적용하지 않으며 자립준비청년과 보호연장아동을 우선 지원합니다.",
182         "alwServCn": "1회 50분 전문 심리상담 10회 제공",
183         "sprtAmtCn": "",
184         "applmetCn": "복지로 온라인 신청",
185         "applmetUrl": "https://www.bokjiro.go.kr/",
186         "servDtllink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000520",
187         "enfcBgngYmd": "20260201",
188         "enfcEndYmd": "20261215",
189         "srvPvsNm": "서비스",
190         "inqNum": "60112"
191     },
192     {
193         "servId": "WII0000861",
194         "servNm": "해운대구 저소득 한부모가족 생활안정 지원",
195         "jurMnofNm": "부산광역시 해운대구",
196         "jurOrgNm": "부산광역시 해운대구",
197         "wlfareInfoOutlCn": "한부모가족의 생활 안정과 자녀 양육을 돕기 위한 구 자체 지원사업입니다.",
198         "sprtTrgtCn": "부산광역시 해운대구에 거주하는 한부모가족 세대주",
199         "slctCritCn": "기준 중위소득 60% 이하 가구를 대상으로 하며, 관내 6개월 이상 거주해야 합니다.",
200         "alwServCn": "가구당 연 60만원 생활안정자금 지급",
201         "sprtAmtCn": "연 60만원",
202         "applmetCn": "해운대구청 가족복지과 방문 신청",
203         "applmetUrl": "https://www.haeundae.go.kr/",
204         "servDtllink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000861",
205         "enfcBgngYmd": "20260301",
206         "enfcEndYmd": "20261031",
207         "srvPvsNm": "현금",
208         "inqNum": "3390"
209     },
210     {
211         "servId": "WII0000970",
212         "servNm": "국민취업지원제도 (I유형)",
213         "jurMnofNm": "고용노동부",
214         "jurOrgNm": "고용노동부",
215         "wlfareInfoOutlCn": "취업을 원하는 구직자에게 취업지원 서비스와 생계 지원을 함께 제공하는 한국형
실업부조입니다.",
216         "sprtTrgtCn": "만 15세 이상 69세 이하 구직자",
217         "slctCritCn": "가구 기준 중위소득 60% 이하이면서 재산 4억원 이하여야 하며, 최근 2년 내 100일 이상 취업 경험이
있어야 합니다.",
218         "alwServCn": "구직촉진수당 월 50만원을 최대 6개월 지급하고 취업지원 서비스를 연계",
219         "sprtAmtCn": "월 50만원",
220         "applmetCn": "국민취업지원제도 누리집 온라인 신청 또는 고용센터 방문",
221         "applmetUrl": "https://www.kua.go.kr/",
222         "servDtllink": "https://www.bokjiro.go.kr/ssis-tbu/twataa/wlfareInfo/moveTWAT52011M.do?
wlfareInfoId=WII0000970",
223         "enfcBgngYmd": "20260101",
224         "enfcEndYmd": "",
225         "srvPvsNm": "현금",
226         "inqNum": "540221"
227     }
228 ]
229 }
230 }

```

```
231     }
232 }
```

5.2 지원사업 공고 응답 구조

ingest/fixtures/bizinfo.json

136 lines · JSON

```
1  {
2    "_fixture_note": "기업마당 지원사업 API 응답 봉투 흉내. 필드명·구조는 W1 실물 검증 대상 (SPEC §3.3). inquireCo(조회수)
   는 휘발성 필드로 content_hash 에서 제외된다 (SPEC §3.2).",
3    "resultCode": "0000",
4    "resultMsg": "정상",
5    "totalCount": 8,
6    "jsonArray": [
7      {
8        "pblancId": "PBLN_000000000098431",
9        "pblancNm": "2026년 소상공인 정책자금 (일반경영안정자금)",
10       "jrdsInstNm": "중소벤처기업부",
11       "excInstNm": "소상공인시장진흥공단",
12       "bsnsSumryCn": "경영 애로를 겪는 소상공인에게 운전자금을 저리로 직접 대출하여 경영 안정을 돕습니다. 대출한도는 업체당 최대
   7천만원이며 금리는 정책자금 기준금리에 연동됩니다.",
13       "trgetNm": "상시근로자 5인 미만 소상공인 또는 자영업자로서 사업자등록을 마친 자. 업력 3년 이상 업체를 우선 지원하며,
   신용등급 하위 구간은 별도 심사합니다.",
14       "pldirSportRealmLclasCodeNm": "금융",
15       "reqstBeginEndDe": "20260105 ~ 예산소진시",
16       "sportScvlNm": "업체당 최대 7,000만원",
17       "reqstMthPapersCn": "소상공인정책자금 누리집 온라인 신청 후 서류 제출",
18       "rceptEnghmpgUrl": "https://ols.semas.or.kr/",
19       "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
   pblancId=PBLN_000000000098431",
20       "inquireCo": "128441",
21       "creatPnttm": "2026-01-05 09:00:00"
22     },
23     {
24       "pblancId": "PBLN_000000000099022",
25       "pblancNm": "부산 지역신용보증재단 소상공인 특례보증",
26       "jrdsInstNm": "부산광역시",
27       "excInstNm": "부산신용보증재단",
28       "bsnsSumryCn": "담보력이 부족한 지역 소상공인의 은행 대출에 대해 보증서를 발급하여 자금 조달을 돕는 사업입니다. 보증료
   일부를 부산광역시가 지원합니다.",
29       "trgetNm": "부산광역시 해운대구에 사업장을 둔 소상공인. 사업자등록 후 6개월 이상 경과하고 관내 6개월 이상 계속 거주 또는
   사업장을 유지해야 합니다.",
30       "pldirSportRealmLclasCodeNm": "금융",
31       "reqstBeginEndDe": "20260201 ~ 20261231",
32       "sportScvlNm": "업체당 최대 5,000만원",
33       "reqstMthPapersCn": "부산신용보증재단 영업점 방문 접수",
34       "rceptEnghmpgUrl": "https://www.busan shinbo.or.kr/",
35       "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
   pblancId=PBLN_000000000099022",
36       "inquireCo": "9120",
37       "creatPnttm": "2026-02-01 09:00:00"
38     },
39     {
40       "pblancId": "PBLN_000000000097355",
41       "pblancNm": "노란우산공제 신규가입 장려금 지원",
42       "jrdsInstNm": "중소벤처기업부",
43       "excInstNm": "중소기업중앙회",
44       "bsnsSumryCn": "폐업·노령 등에 대비한 소상공인 공제 가입을 촉진하기 위해 신규 가입자에게 장려금을 지급합니다.",
45       "trgetNm": "노란우산공제에 신규 가입한 소상공인 및 개인사업자",
46       "pldirSportRealmLclasCodeNm": "금융",
47       "reqstBeginEndDe": "20260101 ~ 20261231",
48       "sportScvlNm": "월 2만원씩 12개월",
49       "reqstMthPapersCn": "노란우산 누리집 온라인 신청",
50       "rceptEnghmpgUrl": "https://www.8899.or.kr/",
```

```

51      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000097355",
52      "inqireCo": "31204",
53      "creatPnttm": "2026-01-02 09:00:00"
54  },
55  {
56      "pblancId": "PBLN_000000000098770",
57      "pblancNm": "청년전용 창업기업 지원자금",
58      "jrdsInsttNm": "중소벤처기업부",
59      "excInsttNm": "중소벤처기업진흥공단",
60      "bsnsSumryCn": "우수한 아이디어를 가진 청년 창업자에게 창업 초기 운전자금과 시설자금을 융자 지원합니다.",
61      "trgetNm": "만 39세 이하 예비창업자 또는 창업 3년 이내 기업의 대표자. 창업 교육 이수자에 한하여 신청할 수 있습니다.",
62      "pldirSportRealmLclasCodeNm": "창업",
63      "reqstBeginEndDe": "20260310 ~ 20260430",
64      "sportScvlNm": "기업당 최대 1억원",
65      "reqstMthPapersCn": "중소벤처기업진흥공단 온라인 신청",
66      "rceptEnghmpgUrl": "https://www.kosmes.or.kr/",
67      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000098770",
68      "inqireCo": "48800",
69      "creatPnttm": "2026-03-10 09:00:00"
70  },
71  {
72      "pblancId": "PBLN_000000000099410",
73      "pblancNm": "전통시장 상인 경영개선 컨설팅",
74      "jrdsInsttNm": "중소벤처기업부",
75      "excInsttNm": "소상공인시장진흥공단",
76      "bsnsSumryCn": "전통시장 점포의 매출 부진 원인을 진단하고 상품 구성, 진열, 홍보 전략을 컨설팅합니다.",
77      "trgetNm": "전통시장 및 상점가 내에서 영업 중인 소상공인",
78      "pldirSportRealmLclasCodeNm": "경영",
79      "reqstBeginEndDe": "20260401 ~ 20261120",
80      "sportScvlNm": "점포당 최대 300만원 상당",
81      "reqstMthPapersCn": "소상공인시장진흥공단 지역센터 방문 접수",
82      "rceptEnghmpgUrl": "https://www.semas.or.kr/",
83      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000099410",
84      "inqireCo": "7412",
85      "creatPnttm": "2026-04-01 09:00:00"
86  },
87  {
88      "pblancId": "PBLN_000000000096001",
89      "pblancNm": "여성기업 확인 및 판로지원 사업",
90      "jrdsInsttNm": "중소벤처기업부",
91      "excInsttNm": "한국여성경제인협회",
92      "bsnsSumryCn": "여성기업 확인서를 발급하고 공공기관 우선구매 및 전시회 참가를 지원합니다.",
93      "trgetNm": "여성기업 확인서를 보유하고 있거나 발급 신청 중인 중소기업. 대표자 성별과 무관하게 법인 요건을 충족하면 신청할 수
있습니다.",
94      "pldirSportRealmLclasCodeNm": "판로",
95      "reqstBeginEndDe": "20260201 ~ 20261130",
96      "sportScvlNm": "기업당 최대 500만원",
97      "reqstMthPapersCn": "한국여성경제인협회 온라인 신청",
98      "rceptEnghmpgUrl": "https://www.womanbiz.or.kr/",
99      "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000096001",
100     "inqireCo": "5321",
101     "creatPnttm": "2026-02-01 09:00:00"
102  },
103  {
104     "pblancId": "PBLN_000000000099880",
105     "pblancNm": "스마트상점 기술보급 사업",
106     "jrdsInsttNm": "중소벤처기업부",
107     "excInsttNm": "소상공인시장진흥공단",
108     "bsnsSumryCn": "키오스크, 스마트 주문 시스템 등 디지털 기술 도입 비용의 일부를 지원해 소상공인의 운영 효율을 높입니다.",
109     "trgetNm": "사업자등록을 마친 소상공인으로 오프라인 점포를 운영 중인 자",
110     "pldirSportRealmLclasCodeNm": "기술",
111     "reqstBeginEndDe": "20260415 ~ 20260630",
112     "sportScvlNm": "점포당 최대 500만원",

```

```

113     "reqstMthPapersCn": "스마트상점 누리집 온라인 신청",
114     "rceptEngnHmpgUrl": "https://www.smartsangjum.kr/",
115     "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000099880",
116     "inqireCo": "22190",
117     "creatPnttm": "2026-04-15 09:00:00"
118 },
119 {
120     "pblancId": "PBLN_000000000095220",
121     "pblancNm": "재도전 성공패키지 (재창업 지원)",
122     "jrdsInsttNm": "중소벤처기업부",
123     "excInsttNm": "창업진흥원",
124     "bsnsSumryCn": "폐업 경험이 있는 재창업자에게 사업화 자금과 교육, 멘토링을 함께 제공합니다.",
125     "trgtcrNm": "예비 또는 3년 이내 재창업자. 폐업 이력이 확인되어야 하며 채무불이행 상태인 경우 신청이 제한됩니다.",
126     "pldirSportRealmLcLasCodeNm": "창업",
127     "reqstBeginEndDe": "20260220 ~ 20260331",
128     "sportScvlNm": "최대 6,000만원",
129     "reqstMthPapersCn": "K-스타트업 누리집 온라인 신청",
130     "rceptEngnHmpgUrl": "https://www.k-startup.go.kr/",
131     "pblancUrl": "https://www.bizinfo.go.kr/web/lay1/bbs/S1T122C128/AS/74/view.do?
pblancId=PBLN_000000000095220",
132     "inqireCo": "16740",
133     "creatPnttm": "2026-02-20 09:00:00"
134 }
135 ]
136 }

```

5.3 금융상품 공시 응답 구조

ingest/fixtures/finlife.json

135 lines · JSON

```

1  {
2    "_fixture_note": "금감원 finlife 예·적금 공시 API 응답 봉투 흉내. baseList + optionList 를 fin_prdt_cd 로
    조인하는 구조까지 재현했다. 필드명은 W1 실물 검증 대상 (SPEC §3.3). fin_co_subm_day 는 매 공시 갱신되는 휘발성 필드라
    content_hash 에서 제외된다.",
3    "result": {
4      "prdt_div": "S",
5      "total_count": 7,
6      "max_page_no": 1,
7      "now_page_no": 1,
8      "err_cd": "000",
9      "err_msg": "정상처리 되었습니다.",
10     "baseList": [
11       {
12         "dcls_month": "202608",
13         "fin_co_no": "0010001",
14         "fin_prdt_cd": "KB-YOUTH-01",
15         "kor_co_nm": "국민은행",
16         "fin_prdt_nm": "KB 청년도약 우대적금",
17         "join_way": "인터넷, 스마트폰, 영업점",
18         "mtrt_int": "만기 후 1년 이내 약정이율의 50%",
19         "spcl_cnd": "급여이체 실적 보유 시 연 0.5%p, 자동이체 6회 이상 시 연 0.3%p 우대",
20         "join_deny": "1",
21         "join_member": "만 19세 이상 34세 이하 실명의 개인. 기준 중위소득 180% 이하 가구에 한합니다.",
22         "etc_note": "1인 1계좌, 월 납입한도 70만원",
23         "max_limit": "70000000",
24         "dcls_strt_day": "20260801",
25         "fin_co_subm_day": "202608031030"
26       },
27       {
28         "dcls_month": "202608",
29         "fin_co_no": "0010002",
30         "fin_prdt_cd": "IBK-BOSS-02",
31         "kor_co_nm": "기업은행",

```

```

32     "fin_prdt_nm": "IBK 사장님 우대적금",
33     "join_way": "영업점,스마트폰",
34     "mtrt_int": "만기 후 기본이율 적용",
35     "spcl_cnd": "노란우산공제 가입 고객 연 0.4%p, 카드 가맹점 실적 보유 시 연 0.3%p 우대",
36     "join_deny": "1",
37     "join_member": "사업자등록증을 보유한 개인사업자 또는 소상공인",
38     "etc_note": "월 납입한도 300만원, 계약기간 12~36개월",
39     "max_limit": "100000000",
40     "dcls_strt_day": "20260801",
41     "fin_co_subm_day": "202608031102"
42 },
43 {
44     "dcls_month": "202608",
45     "fin_co_no": "0010003",
46     "fin_prdt_cd": "SHB-EDEP-05",
47     "kor_co_nm": "신한은행",
48     "fin_prdt_nm": "신한 e-정기예금",
49     "join_way": "인터넷,스마트폰",
50     "mtrt_int": "만기 후 1개월 이내 약정이율의 50%",
51     "spcl_cnd": "해당사항 없음",
52     "join_deny": "1",
53     "join_member": "실명의 개인 및 개인사업자",
54     "etc_note": "최소 가입금액 100만원",
55     "max_limit": "500000000",
56     "dcls_strt_day": "20260801",
57     "fin_co_subm_day": "202608030940"
58 },
59 {
60     "dcls_month": "202608",
61     "fin_co_no": "0010004",
62     "fin_prdt_cd": "NH-SENIOR-03",
63     "kor_co_nm": "농협은행",
64     "fin_prdt_nm": "NH 시니어 행복적금",
65     "join_way": "영업점,인터넷",
66     "mtrt_int": "만기 후 기본이율의 50%",
67     "spcl_cnd": "연금 수령 실적 보유 시 연 0.6%p, 농업인 확인서 제출 시 연 0.2%p 우대",
68     "join_deny": "1",
69     "join_member": "만 60세 이상 실명의 개인",
70     "etc_note": "월 납입한도 50만원",
71     "max_limit": "300000000",
72     "dcls_strt_day": "20260801",
73     "fin_co_subm_day": "202608041500"
74 },
75 {
76     "dcls_month": "202608",
77     "fin_co_no": "0010005",
78     "fin_prdt_cd": "HANA-FREE-07",
79     "kor_co_nm": "하나은행",
80     "fin_prdt_nm": "하나 청년 자유적금",
81     "join_way": "스마트폰",
82     "mtrt_int": "만기 후 1년 이내 약정이율의 30%",
83     "spcl_cnd": "첫 거래 고객 연 0.5%p, 마케팅 동의 시 연 0.1%p 우대",
84     "join_deny": "1",
85     "join_member": "만 19세 이상 34세 이하 실명의 개인",
86     "etc_note": "자유적립식, 월 최대 50만원",
87     "max_limit": "180000000",
88     "dcls_strt_day": "20260801",
89     "fin_co_subm_day": "202608031345"
90 },
91 {
92     "dcls_month": "202608",
93     "fin_co_no": "0010006",
94     "fin_prdt_cd": "WOORI-HOUSE-09",
95     "kor_co_nm": "우리은행",
96     "fin_prdt_nm": "우리 주택청약 우대 정기예금",
97     "join_way": "영업점,인터넷,스마트폰",
98     "mtrt_int": "만기 후 기본이율 적용",

```

```

99     "spcl_cnd": "무주택 세대주 확인서 제출 시 연 0.4%p, 청약저축 보유 시 연 0.2%p 우대",
100     "join_deny": "1",
101     "join_member": "실명의 개인",
102     "etc_note": "가입금액 300만원 이상",
103     "max_limit": "300000000",
104     "dcls_strt_day": "20260801",
105     "fin_co_subm_day": "202608030855"
106 },
107 {
108     "dcls_month": "202608",
109     "fin_co_no": "0010007",
110     "fin_prdt_cd": "KAKAO-PARK-11",
111     "kor_co_nm": "카카오뱅크",
112     "fin_prdt_nm": "카카오뱅크 세이프박스",
113     "join_way": "스마트폰",
114     "mtrt_int": "해당사항 없음",
115     "spcl_cnd": "해당사항 없음",
116     "join_deny": "1",
117     "join_member": "실명의 개인",
118     "etc_note": "입출금 자유, 한도 내 일복리",
119     "max_limit": "100000000",
120     "dcls_strt_day": "20260801",
121     "fin_co_subm_day": "202608031201"
122 }
123 ],
124 "optionList": [
125     { "fin_co_no": "0010001", "fin_prdt_cd": "KB-YOUTH-01", "intr_rate_type_nm": "단리", "save_trm":
126 "12", "intr_rate": 3.5, "intr_rate2": 4.8 },
127     { "fin_co_no": "0010001", "fin_prdt_cd": "KB-YOUTH-01", "intr_rate_type_nm": "단리", "save_trm":
128 "24", "intr_rate": 3.7, "intr_rate2": 5.0 },
129     { "fin_co_no": "0010002", "fin_prdt_cd": "IBK-BOSS-02", "intr_rate_type_nm": "단리", "save_trm":
130 "12", "intr_rate": 3.2, "intr_rate2": 3.9 },
131     { "fin_co_no": "0010003", "fin_prdt_cd": "SHB-EDEP-05", "intr_rate_type_nm": "단리", "save_trm":
132 "12", "intr_rate": 3.0, "intr_rate2": 3.0 },
133     { "fin_co_no": "0010004", "fin_prdt_cd": "NH-SENIOR-03", "intr_rate_type_nm": "단리", "save_trm":
134 "12", "intr_rate": 3.3, "intr_rate2": 4.1 },
135     { "fin_co_no": "0010005", "fin_prdt_cd": "HANA-FREE-07", "intr_rate_type_nm": "단리", "save_trm":
136 "12", "intr_rate": 3.1, "intr_rate2": 3.7 },
137     { "fin_co_no": "0010006", "fin_prdt_cd": "WOORI-HOUSE-09", "intr_rate_type_nm": "단리", "save_trm":
138 "12", "intr_rate": 2.9, "intr_rate2": 3.5 },
139     { "fin_co_no": "0010007", "fin_prdt_cd": "KAKAO-PARK-11", "intr_rate_type_nm": "단리", "save_trm":
140 "0", "intr_rate": 2.0, "intr_rate2": 2.0 }
141 ]
142 }
143 }

```


6. 자격 판정 기준 데이터 (JSON)

API 응답의 서술형 자격요건을 정형 조건으로 변환할 때 사용하는 기준 데이터입니다. 행정표준코드(법정동코드)와 통계청 자료를 근거로 구성했습니다.

6.1 지역 코드

shared/regions.json1364 lines · JSON

```
1 {
2   "_note": "행정표준코드 전체자료(2026-07-08) 기반 17개 시도·256개 시군구. 비자치 상위시는 제외하고 광주·전남 호환 코드를
3   유지한다.",
4   "source": {
5     "issuer": "행정안전부",
6     "name": "행정표준코드 법정동코드 전체자료",
7     "url": "https://www.code.go.kr/stdcode/regCodeL.do",
8     "snapshot_date": "2026-07-08",
9     "_why_static": "연 단위로만 바뀌므로 API 연동 대신 1회 다운로드 후 정적 파일로 둔다 (SPEC §3.3 T1-인프라).",
10    "_update_trigger": "시군구 통폐합·관할 이관·특별자치도 전환이 고시되면 갱신한다. 낡은 표를 쓰면 사용자 지역코드와 공고
11    지역코드가 조용히 불일치해 정당한 대상자가 탈락한다.",
12    "_verification": "web/src/lib/regions.test.mjs 가 자리수·소속·중복과 최근 개편(강원 51 / 전북 52 / 군위군 대구
13    이관) 반영 여부를 검사한다."
14  },
15  "sido": [
16    {
17      "code": "11",
18      "name": "서울특별시"
19    },
20    {
21      "code": "26",
22      "name": "부산광역시"
23    },
24    {
25      "code": "27",
26      "name": "대구광역시"
27    },
28    {
29      "code": "28",
30      "name": "인천광역시"
31    },
32    {
33      "code": "29",
34      "name": "광주광역시"
35    },
36    {
37      "code": "30",
38      "name": "대전광역시"
39    },
40    {
41      "code": "31",
42      "name": "울산광역시"
43    },
44    {
45      "code": "36",
46      "name": "세종특별자치시"
47    },
48    {
49      "code": "41",
50      "name": "경기도"
```

```
51     "name": "충청북도"
52 },
53 {
54     "code": "44",
55     "name": "충청남도"
56 },
57 {
58     "code": "46",
59     "name": "전라남도"
60 },
61 {
62     "code": "47",
63     "name": "경상북도"
64 },
65 {
66     "code": "48",
67     "name": "경상남도"
68 },
69 {
70     "code": "50",
71     "name": "제주특별자치도"
72 },
73 {
74     "code": "51",
75     "name": "강원특별자치도"
76 },
77 {
78     "code": "52",
79     "name": "전북특별자치도"
80 }
81 ],
82 "sigungu": [
83     {
84         "code": "11110",
85         "name": "종로구",
86         "sido": "11"
87     },
88     {
89         "code": "11140",
90         "name": "중구",
91         "sido": "11"
92     },
93     {
94         "code": "11170",
95         "name": "용산구",
96         "sido": "11"
97     },
98     {
99         "code": "11200",
100        "name": "성동구",
101        "sido": "11"
102    },
103    {
104        "code": "11215",
105        "name": "광진구",
106        "sido": "11"
107    },
108    {
109        "code": "11230",
110        "name": "동대문구",
111        "sido": "11"
112    },
113    {
114        "code": "11260",
115        "name": "종랑구",
116        "sido": "11"
117    },
118 }
```

```
118 {
119     "code": "11290",
120     "name": "성북구",
121     "sido": "11"
122 },
123 {
124     "code": "11305",
125     "name": "강북구",
126     "sido": "11"
127 },
128 {
129     "code": "11320",
130     "name": "도봉구",
131     "sido": "11"
132 },
133 {
134     "code": "11350",
135     "name": "노원구",
136     "sido": "11"
137 },
138 {
139     "code": "11380",
140     "name": "은평구",
141     "sido": "11"
142 },
143 {
144     "code": "11410",
145     "name": "서대문구",
146     "sido": "11"
147 },
148 {
149     "code": "11440",
150     "name": "마포구",
151     "sido": "11"
152 },
153 {
154     "code": "11470",
155     "name": "양천구",
156     "sido": "11"
157 },
158 {
159     "code": "11500",
160     "name": "강서구",
161     "sido": "11"
162 },
163 {
164     "code": "11530",
165     "name": "구로구",
166     "sido": "11"
167 },
168 {
169     "code": "11545",
170     "name": "금천구",
171     "sido": "11"
172 },
173 {
174     "code": "11560",
175     "name": "영등포구",
176     "sido": "11"
177 },
178 {
179     "code": "11590",
180     "name": "동작구",
181     "sido": "11"
182 },
183 {
184     "code": "11620",
```

```
185     "name": "관악구",
186     "sido": "11"
187 },
188 {
189     "code": "11650",
190     "name": "서초구",
191     "sido": "11"
192 },
193 {
194     "code": "11680",
195     "name": "강남구",
196     "sido": "11"
197 },
198 {
199     "code": "11710",
200     "name": "송파구",
201     "sido": "11"
202 },
203 {
204     "code": "11740",
205     "name": "강동구",
206     "sido": "11"
207 },
208 {
209     "code": "26110",
210     "name": "종구",
211     "sido": "26"
212 },
213 {
214     "code": "26140",
215     "name": "서구",
216     "sido": "26"
217 },
218 {
219     "code": "26170",
220     "name": "동구",
221     "sido": "26"
222 },
223 {
224     "code": "26200",
225     "name": "영도구",
226     "sido": "26"
227 },
228 {
229     "code": "26230",
230     "name": "부산진구",
231     "sido": "26"
232 },
233 {
234     "code": "26260",
235     "name": "동래구",
236     "sido": "26"
237 },
238 {
239     "code": "26290",
240     "name": "남구",
241     "sido": "26"
242 },
243 {
244     "code": "26320",
245     "name": "북구",
246     "sido": "26"
247 },
248 {
249     "code": "26350",
250     "name": "해운대구",
251     "sido": "26"
```

```
252 },
253 {
254     "code": "26380",
255     "name": "사하구",
256     "sido": "26"
257 },
258 {
259     "code": "26410",
260     "name": "금정구",
261     "sido": "26"
262 },
263 {
264     "code": "26440",
265     "name": "강서구",
266     "sido": "26"
267 },
268 {
269     "code": "26470",
270     "name": "연제구",
271     "sido": "26"
272 },
273 {
274     "code": "26500",
275     "name": "수영구",
276     "sido": "26"
277 },
278 {
279     "code": "26530",
280     "name": "사상구",
281     "sido": "26"
282 },
283 {
284     "code": "26710",
285     "name": "기장군",
286     "sido": "26"
287 },
288 {
289     "code": "27110",
290     "name": "중구",
291     "sido": "27"
292 },
293 {
294     "code": "27140",
295     "name": "동구",
296     "sido": "27"
297 },
298 {
299     "code": "27170",
300     "name": "서구",
301     "sido": "27"
302 },
303 {
304     "code": "27200",
305     "name": "남구",
306     "sido": "27"
307 },
308 {
309     "code": "27230",
310     "name": "북구",
311     "sido": "27"
312 },
313 {
314     "code": "27260",
315     "name": "수성구",
316     "sido": "27"
317 },
318 {
```

```
319     "code": "27290",
320     "name": "달서구",
321     "sido": "27"
322 },
323 {
324     "code": "27710",
325     "name": "달성군",
326     "sido": "27"
327 },
328 {
329     "code": "27720",
330     "name": "군위군",
331     "sido": "27"
332 },
333 {
334     "code": "28125",
335     "name": "제물포구",
336     "sido": "28"
337 },
338 {
339     "code": "28155",
340     "name": "영종구",
341     "sido": "28"
342 },
343 {
344     "code": "28177",
345     "name": "미추홀구",
346     "sido": "28"
347 },
348 {
349     "code": "28185",
350     "name": "연수구",
351     "sido": "28"
352 },
353 {
354     "code": "28200",
355     "name": "남동구",
356     "sido": "28"
357 },
358 {
359     "code": "28237",
360     "name": "부평구",
361     "sido": "28"
362 },
363 {
364     "code": "28245",
365     "name": "계양구",
366     "sido": "28"
367 },
368 {
369     "code": "28275",
370     "name": "서해구",
371     "sido": "28"
372 },
373 {
374     "code": "28290",
375     "name": "검단구",
376     "sido": "28"
377 },
378 {
379     "code": "28710",
380     "name": "강화군",
381     "sido": "28"
382 },
383 {
384     "code": "28720",
385     "name": "옹진군",
```

```
386     "sido": "28"
387 },
388 {
389     "code": "29110",
390     "name": "동구",
391     "sido": "29"
392 },
393 {
394     "code": "29140",
395     "name": "서구",
396     "sido": "29"
397 },
398 {
399     "code": "29155",
400     "name": "남구",
401     "sido": "29"
402 },
403 {
404     "code": "29170",
405     "name": "북구",
406     "sido": "29"
407 },
408 {
409     "code": "29200",
410     "name": "광산구",
411     "sido": "29"
412 },
413 {
414     "code": "30110",
415     "name": "동구",
416     "sido": "30"
417 },
418 {
419     "code": "30140",
420     "name": "중구",
421     "sido": "30"
422 },
423 {
424     "code": "30170",
425     "name": "서구",
426     "sido": "30"
427 },
428 {
429     "code": "30200",
430     "name": "유성구",
431     "sido": "30"
432 },
433 {
434     "code": "30230",
435     "name": "대덕구",
436     "sido": "30"
437 },
438 {
439     "code": "31110",
440     "name": "중구",
441     "sido": "31"
442 },
443 {
444     "code": "31140",
445     "name": "남구",
446     "sido": "31"
447 },
448 {
449     "code": "31170",
450     "name": "동구",
451     "sido": "31"
452 },
```

```
453 {
454     "code": "31200",
455     "name": "북구",
456     "sido": "31"
457 },
458 {
459     "code": "31710",
460     "name": "울주군",
461     "sido": "31"
462 },
463 {
464     "code": "36110",
465     "name": "세종시",
466     "sido": "36"
467 },
468 {
469     "code": "41111",
470     "name": "수원시 장안구",
471     "sido": "41"
472 },
473 {
474     "code": "41113",
475     "name": "수원시 권선구",
476     "sido": "41"
477 },
478 {
479     "code": "41115",
480     "name": "수원시 팔달구",
481     "sido": "41"
482 },
483 {
484     "code": "41117",
485     "name": "수원시 영통구",
486     "sido": "41"
487 },
488 {
489     "code": "41131",
490     "name": "성남시 수정구",
491     "sido": "41"
492 },
493 {
494     "code": "41133",
495     "name": "성남시 중원구",
496     "sido": "41"
497 },
498 {
499     "code": "41135",
500     "name": "성남시 분당구",
501     "sido": "41"
502 },
503 {
504     "code": "41150",
505     "name": "의정부시",
506     "sido": "41"
507 },
508 {
509     "code": "41171",
510     "name": "안양시 만안구",
511     "sido": "41"
512 },
513 {
514     "code": "41173",
515     "name": "안양시 동안구",
516     "sido": "41"
517 },
518 {
519     "code": "41192",
```



```
520     "name": "부천시 원미구",
521     "sido": "41"
522 },
523 {
524     "code": "41194",
525     "name": "부천시 소사구",
526     "sido": "41"
527 },
528 {
529     "code": "41196",
530     "name": "부천시 오정구",
531     "sido": "41"
532 },
533 {
534     "code": "41210",
535     "name": "광명시",
536     "sido": "41"
537 },
538 {
539     "code": "41220",
540     "name": "평택시",
541     "sido": "41"
542 },
543 {
544     "code": "41250",
545     "name": "동두천시",
546     "sido": "41"
547 },
548 {
549     "code": "41271",
550     "name": "안산시 상록구",
551     "sido": "41"
552 },
553 {
554     "code": "41273",
555     "name": "안산시 단원구",
556     "sido": "41"
557 },
558 {
559     "code": "41281",
560     "name": "고양시 덕양구",
561     "sido": "41"
562 },
563 {
564     "code": "41285",
565     "name": "고양시 일산동구",
566     "sido": "41"
567 },
568 {
569     "code": "41287",
570     "name": "고양시 일산서구",
571     "sido": "41"
572 },
573 {
574     "code": "41290",
575     "name": "과천시",
576     "sido": "41"
577 },
578 {
579     "code": "41310",
580     "name": "구리시",
581     "sido": "41"
582 },
583 {
584     "code": "41360",
585     "name": "남양주시",
586     "sido": "41"
```

```
587     },
588     {
589         "code": "41370",
590         "name": "오산시",
591         "sido": "41"
592     },
593     {
594         "code": "41390",
595         "name": "시흥시",
596         "sido": "41"
597     },
598     {
599         "code": "41410",
600         "name": "군포시",
601         "sido": "41"
602     },
603     {
604         "code": "41430",
605         "name": "의왕시",
606         "sido": "41"
607     },
608     {
609         "code": "41450",
610         "name": "하남시",
611         "sido": "41"
612     },
613     {
614         "code": "41461",
615         "name": "용인시 처인구",
616         "sido": "41"
617     },
618     {
619         "code": "41463",
620         "name": "용인시 기흥구",
621         "sido": "41"
622     },
623     {
624         "code": "41465",
625         "name": "용인시 수지구",
626         "sido": "41"
627     },
628     {
629         "code": "41480",
630         "name": "파주시",
631         "sido": "41"
632     },
633     {
634         "code": "41500",
635         "name": "이천시",
636         "sido": "41"
637     },
638     {
639         "code": "41550",
640         "name": "안성시",
641         "sido": "41"
642     },
643     {
644         "code": "41570",
645         "name": "김포시",
646         "sido": "41"
647     },
648     {
649         "code": "41591",
650         "name": "화성시 만세구",
651         "sido": "41"
652     },
653     {
```

```
654     "code": "41593",
655     "name": "화성시 효행구",
656     "sido": "41"
657 },
658 {
659     "code": "41595",
660     "name": "화성시 병점구",
661     "sido": "41"
662 },
663 {
664     "code": "41597",
665     "name": "화성시 동탄구",
666     "sido": "41"
667 },
668 {
669     "code": "41610",
670     "name": "광주시",
671     "sido": "41"
672 },
673 {
674     "code": "41630",
675     "name": "양주시",
676     "sido": "41"
677 },
678 {
679     "code": "41650",
680     "name": "포천시",
681     "sido": "41"
682 },
683 {
684     "code": "41670",
685     "name": "여주시",
686     "sido": "41"
687 },
688 {
689     "code": "41800",
690     "name": "연천군",
691     "sido": "41"
692 },
693 {
694     "code": "41820",
695     "name": "가평군",
696     "sido": "41"
697 },
698 {
699     "code": "41830",
700     "name": "양평군",
701     "sido": "41"
702 },
703 {
704     "code": "43111",
705     "name": "청주시 상당구",
706     "sido": "43"
707 },
708 {
709     "code": "43112",
710     "name": "청주시 서원구",
711     "sido": "43"
712 },
713 {
714     "code": "43113",
715     "name": "청주시 흥덕구",
716     "sido": "43"
717 },
718 {
719     "code": "43114",
720     "name": "청주시 청원구",
```

```
721     "sido": "43"
722 },
723 {
724     "code": "43130",
725     "name": "충주시",
726     "sido": "43"
727 },
728 {
729     "code": "43150",
730     "name": "제천시",
731     "sido": "43"
732 },
733 {
734     "code": "43720",
735     "name": "보은군",
736     "sido": "43"
737 },
738 {
739     "code": "43730",
740     "name": "옥천군",
741     "sido": "43"
742 },
743 {
744     "code": "43740",
745     "name": "영동군",
746     "sido": "43"
747 },
748 {
749     "code": "43745",
750     "name": "증평군",
751     "sido": "43"
752 },
753 {
754     "code": "43750",
755     "name": "진천군",
756     "sido": "43"
757 },
758 {
759     "code": "43760",
760     "name": "괴산군",
761     "sido": "43"
762 },
763 {
764     "code": "43770",
765     "name": "음성군",
766     "sido": "43"
767 },
768 {
769     "code": "43800",
770     "name": "단양군",
771     "sido": "43"
772 },
773 {
774     "code": "44131",
775     "name": "천안시 동남구",
776     "sido": "44"
777 },
778 {
779     "code": "44133",
780     "name": "천안시 서북구",
781     "sido": "44"
782 },
783 {
784     "code": "44150",
785     "name": "공주시",
786     "sido": "44"
787 },
```

```
788 {
789     "code": "44180",
790     "name": "보령시",
791     "sido": "44"
792 },
793 {
794     "code": "44200",
795     "name": "아산시",
796     "sido": "44"
797 },
798 {
799     "code": "44210",
800     "name": "서산시",
801     "sido": "44"
802 },
803 {
804     "code": "44230",
805     "name": "논산시",
806     "sido": "44"
807 },
808 {
809     "code": "44250",
810     "name": "계룡시",
811     "sido": "44"
812 },
813 {
814     "code": "44270",
815     "name": "당진시",
816     "sido": "44"
817 },
818 {
819     "code": "44710",
820     "name": "금산군",
821     "sido": "44"
822 },
823 {
824     "code": "44760",
825     "name": "부여군",
826     "sido": "44"
827 },
828 {
829     "code": "44770",
830     "name": "서천군",
831     "sido": "44"
832 },
833 {
834     "code": "44790",
835     "name": "청양군",
836     "sido": "44"
837 },
838 {
839     "code": "44800",
840     "name": "홍성군",
841     "sido": "44"
842 },
843 {
844     "code": "44810",
845     "name": "예산군",
846     "sido": "44"
847 },
848 {
849     "code": "44825",
850     "name": "태안군",
851     "sido": "44"
852 },
853 {
854     "code": "46110",
```

```
855     "name": "목포시",
856     "sido": "46"
857 },
858 {
859     "code": "46130",
860     "name": "여주시",
861     "sido": "46"
862 },
863 {
864     "code": "46150",
865     "name": "순천시",
866     "sido": "46"
867 },
868 {
869     "code": "46170",
870     "name": "나주시",
871     "sido": "46"
872 },
873 {
874     "code": "46230",
875     "name": "광양시",
876     "sido": "46"
877 },
878 {
879     "code": "46710",
880     "name": "담양군",
881     "sido": "46"
882 },
883 {
884     "code": "46720",
885     "name": "곡성군",
886     "sido": "46"
887 },
888 {
889     "code": "46730",
890     "name": "구례군",
891     "sido": "46"
892 },
893 {
894     "code": "46770",
895     "name": "고흥군",
896     "sido": "46"
897 },
898 {
899     "code": "46780",
900     "name": "보성군",
901     "sido": "46"
902 },
903 {
904     "code": "46790",
905     "name": "화순군",
906     "sido": "46"
907 },
908 {
909     "code": "46800",
910     "name": "장흥군",
911     "sido": "46"
912 },
913 {
914     "code": "46810",
915     "name": "강진군",
916     "sido": "46"
917 },
918 {
919     "code": "46820",
920     "name": "해남군",
921     "sido": "46"
```

```
922 },
923 {
924     "code": "46830",
925     "name": "영암군",
926     "sido": "46"
927 },
928 {
929     "code": "46840",
930     "name": "무안군",
931     "sido": "46"
932 },
933 {
934     "code": "46860",
935     "name": "함평군",
936     "sido": "46"
937 },
938 {
939     "code": "46870",
940     "name": "영광군",
941     "sido": "46"
942 },
943 {
944     "code": "46880",
945     "name": "장성군",
946     "sido": "46"
947 },
948 {
949     "code": "46890",
950     "name": "완도군",
951     "sido": "46"
952 },
953 {
954     "code": "46900",
955     "name": "진도군",
956     "sido": "46"
957 },
958 {
959     "code": "46910",
960     "name": "신안군",
961     "sido": "46"
962 },
963 {
964     "code": "47111",
965     "name": "포항시 남구",
966     "sido": "47"
967 },
968 {
969     "code": "47113",
970     "name": "포항시 북구",
971     "sido": "47"
972 },
973 {
974     "code": "47130",
975     "name": "경주시",
976     "sido": "47"
977 },
978 {
979     "code": "47150",
980     "name": "김천시",
981     "sido": "47"
982 },
983 {
984     "code": "47170",
985     "name": "안동시",
986     "sido": "47"
987 },
988 {
```

```
989     "code": "47190",
990     "name": "구미시",
991     "sido": "47"
992 },
993 {
994     "code": "47210",
995     "name": "영주시",
996     "sido": "47"
997 },
998 {
999     "code": "47230",
1000     "name": "영천시",
1001     "sido": "47"
1002 },
1003 {
1004     "code": "47250",
1005     "name": "상주시",
1006     "sido": "47"
1007 },
1008 {
1009     "code": "47280",
1010     "name": "문경시",
1011     "sido": "47"
1012 },
1013 {
1014     "code": "47290",
1015     "name": "경산시",
1016     "sido": "47"
1017 },
1018 {
1019     "code": "47730",
1020     "name": "의성군",
1021     "sido": "47"
1022 },
1023 {
1024     "code": "47750",
1025     "name": "청송군",
1026     "sido": "47"
1027 },
1028 {
1029     "code": "47760",
1030     "name": "영양군",
1031     "sido": "47"
1032 },
1033 {
1034     "code": "47770",
1035     "name": "영덕군",
1036     "sido": "47"
1037 },
1038 {
1039     "code": "47820",
1040     "name": "청도군",
1041     "sido": "47"
1042 },
1043 {
1044     "code": "47830",
1045     "name": "고령군",
1046     "sido": "47"
1047 },
1048 {
1049     "code": "47840",
1050     "name": "성주군",
1051     "sido": "47"
1052 },
1053 {
1054     "code": "47850",
1055     "name": "칠곡군",
```



```
1056     "sido": "47"
1057 },
1058 {
1059     "code": "47900",
1060     "name": "예천군",
1061     "sido": "47"
1062 },
1063 {
1064     "code": "47920",
1065     "name": "봉화군",
1066     "sido": "47"
1067 },
1068 {
1069     "code": "47930",
1070     "name": "울진군",
1071     "sido": "47"
1072 },
1073 {
1074     "code": "47940",
1075     "name": "울릉군",
1076     "sido": "47"
1077 },
1078 {
1079     "code": "48121",
1080     "name": "창원시 의창구",
1081     "sido": "48"
1082 },
1083 {
1084     "code": "48123",
1085     "name": "창원시 성산구",
1086     "sido": "48"
1087 },
1088 {
1089     "code": "48125",
1090     "name": "창원시 마산합포구",
1091     "sido": "48"
1092 },
1093 {
1094     "code": "48127",
1095     "name": "창원시 마산회원구",
1096     "sido": "48"
1097 },
1098 {
1099     "code": "48129",
1100     "name": "창원시 진해구",
1101     "sido": "48"
1102 },
1103 {
1104     "code": "48170",
1105     "name": "진주시",
1106     "sido": "48"
1107 },
1108 {
1109     "code": "48220",
1110     "name": "통영시",
1111     "sido": "48"
1112 },
1113 {
1114     "code": "48240",
1115     "name": "사천시",
1116     "sido": "48"
1117 },
1118 {
1119     "code": "48250",
1120     "name": "김해시",
1121     "sido": "48"
1122 },
1123 }
```

```
1123 {
1124     "code": "48270",
1125     "name": "밀양시",
1126     "sido": "48"
1127 },
1128 {
1129     "code": "48310",
1130     "name": "거제시",
1131     "sido": "48"
1132 },
1133 {
1134     "code": "48330",
1135     "name": "양산시",
1136     "sido": "48"
1137 },
1138 {
1139     "code": "48720",
1140     "name": "의령군",
1141     "sido": "48"
1142 },
1143 {
1144     "code": "48730",
1145     "name": "함안군",
1146     "sido": "48"
1147 },
1148 {
1149     "code": "48740",
1150     "name": "창녕군",
1151     "sido": "48"
1152 },
1153 {
1154     "code": "48820",
1155     "name": "고성군",
1156     "sido": "48"
1157 },
1158 {
1159     "code": "48840",
1160     "name": "남해군",
1161     "sido": "48"
1162 },
1163 {
1164     "code": "48850",
1165     "name": "하동군",
1166     "sido": "48"
1167 },
1168 {
1169     "code": "48860",
1170     "name": "산청군",
1171     "sido": "48"
1172 },
1173 {
1174     "code": "48870",
1175     "name": "함양군",
1176     "sido": "48"
1177 },
1178 {
1179     "code": "48880",
1180     "name": "거창군",
1181     "sido": "48"
1182 },
1183 {
1184     "code": "48890",
1185     "name": "합천군",
1186     "sido": "48"
1187 },
1188 {
1189     "code": "50110",
```

```
1190     "name": "제주시",
1191     "sido": "50"
1192 },
1193 {
1194     "code": "50130",
1195     "name": "서귀포시",
1196     "sido": "50"
1197 },
1198 {
1199     "code": "51110",
1200     "name": "춘천시",
1201     "sido": "51"
1202 },
1203 {
1204     "code": "51130",
1205     "name": "원주시",
1206     "sido": "51"
1207 },
1208 {
1209     "code": "51150",
1210     "name": "강릉시",
1211     "sido": "51"
1212 },
1213 {
1214     "code": "51170",
1215     "name": "동해시",
1216     "sido": "51"
1217 },
1218 {
1219     "code": "51190",
1220     "name": "태백시",
1221     "sido": "51"
1222 },
1223 {
1224     "code": "51210",
1225     "name": "속초시",
1226     "sido": "51"
1227 },
1228 {
1229     "code": "51230",
1230     "name": "삼척시",
1231     "sido": "51"
1232 },
1233 {
1234     "code": "51720",
1235     "name": "홍천군",
1236     "sido": "51"
1237 },
1238 {
1239     "code": "51730",
1240     "name": "횡성군",
1241     "sido": "51"
1242 },
1243 {
1244     "code": "51750",
1245     "name": "영월군",
1246     "sido": "51"
1247 },
1248 {
1249     "code": "51760",
1250     "name": "평창군",
1251     "sido": "51"
1252 },
1253 {
1254     "code": "51770",
1255     "name": "정선군",
1256     "sido": "51"
```

```
1257 },
1258 {
1259     "code": "51780",
1260     "name": "철원군",
1261     "sido": "51"
1262 },
1263 {
1264     "code": "51790",
1265     "name": "화천군",
1266     "sido": "51"
1267 },
1268 {
1269     "code": "51800",
1270     "name": "양구군",
1271     "sido": "51"
1272 },
1273 {
1274     "code": "51810",
1275     "name": "인제군",
1276     "sido": "51"
1277 },
1278 {
1279     "code": "51820",
1280     "name": "고성군",
1281     "sido": "51"
1282 },
1283 {
1284     "code": "51830",
1285     "name": "양양군",
1286     "sido": "51"
1287 },
1288 {
1289     "code": "52111",
1290     "name": "전주시 완산구",
1291     "sido": "52"
1292 },
1293 {
1294     "code": "52113",
1295     "name": "전주시 덕진구",
1296     "sido": "52"
1297 },
1298 {
1299     "code": "52130",
1300     "name": "군산시",
1301     "sido": "52"
1302 },
1303 {
1304     "code": "52140",
1305     "name": "익산시",
1306     "sido": "52"
1307 },
1308 {
1309     "code": "52180",
1310     "name": "정읍시",
1311     "sido": "52"
1312 },
1313 {
1314     "code": "52190",
1315     "name": "남원시",
1316     "sido": "52"
1317 },
1318 {
1319     "code": "52210",
1320     "name": "김제시",
1321     "sido": "52"
1322 },
1323 {
```

```

1324     "code": "52710",
1325     "name": "완주군",
1326     "sido": "52"
1327 },
1328 {
1329     "code": "52720",
1330     "name": "진안군",
1331     "sido": "52"
1332 },
1333 {
1334     "code": "52730",
1335     "name": "무주군",
1336     "sido": "52"
1337 },
1338 {
1339     "code": "52740",
1340     "name": "장수군",
1341     "sido": "52"
1342 },
1343 {
1344     "code": "52750",
1345     "name": "임실군",
1346     "sido": "52"
1347 },
1348 {
1349     "code": "52770",
1350     "name": "순창군",
1351     "sido": "52"
1352 },
1353 {
1354     "code": "52790",
1355     "name": "고창군",
1356     "sido": "52"
1357 },
1358 {
1359     "code": "52800",
1360     "name": "부안군",
1361     "sido": "52"
1362 }
1363 ]
1364 }

```

6.2 직업 분류

shared/occupations.json

17 lines · JSON

```

1  {
2    "_note": "직업 대분류 12종 (SPEC §5 입력 필드 3). code는 DB·파서·화면 공통. synonyms는 파서 사전 룩업용 (§6.1).",
3    "categories": [
4      { "code": "employee_office", "name": "사무직·전문직", "synonyms": ["사무직", "회사원", "직장인", "근로자", "노동자", "전문직"] },
5      { "code": "employee_field", "name": "현장·생산·서비스직", "synonyms": ["생산직", "현장직", "서비스직", "기능공", "근로자", "노동자"] },
6      { "code": "self_employed", "name": "자영업·소상공인", "synonyms": ["자영업자", "소상공인", "개인사업자", "사업자", "소기업"] },
7      { "code": "farmer_fisher", "name": "농림어업인", "synonyms": ["농업인", "어업인", "임업인", "농어민", "농가"] },
8    },
9    { "code": "freelancer", "name": "프리랜서·특수고용", "synonyms": ["프리랜서", "특수형태근로종사자", "특고", "플랫폼노동자"] },
10   { "code": "student", "name": "학생", "synonyms": ["대학생", "대학원생", "재학생"] },
11   { "code": "jobseeker", "name": "구직자·무직", "synonyms": ["구직자", "무직", "실업자", "미취업", "취업준비생"] },
12   },
13   { "code": "homemaker", "name": "주부", "synonyms": ["전업주부", "가사"] },
14   { "code": "public_servant", "name": "공무원·군인", "synonyms": ["공무원", "군인", "직업군인"] },

```

```

13     { "code": "medical", "name": "보건·의료", "synonyms": ["의료인", "간호사", "의사", "보건"] },
14     { "code": "education", "name": "교육", "synonyms": ["교사", "교원", "강사", "교직원"] },
15     { "code": "retired", "name": "은퇴·연금생활", "synonyms": ["은퇴자", "퇴직자", "연금수급자", "어르신", "노인"] }
16   ]
17 }

```

6.3 소득분위 기준

shared/income_deciles.json

15 lines · JSON

```

1  {
2    "_note": "소득분위 자가 입력용 라벨 (SPEC §5). 월 소득 구간은 데모용 추정치 - W1에 최신 고시 기준으로 갱신 (연 1회 갱신 정책).",
3    "deciles": [
4      { "decile": 1, "label": "1분위", "monthly_income_hint": "월 소득 약 ~95만원" },
5      { "decile": 2, "label": "2분위", "monthly_income_hint": "월 소득 약 95~160만원" },
6      { "decile": 3, "label": "3분위", "monthly_income_hint": "월 소득 약 160~230만원" },
7      { "decile": 4, "label": "4분위", "monthly_income_hint": "월 소득 약 230~300만원" },
8      { "decile": 5, "label": "5분위", "monthly_income_hint": "월 소득 약 300~370만원" },
9      { "decile": 6, "label": "6분위", "monthly_income_hint": "월 소득 약 370~450만원" },
10     { "decile": 7, "label": "7분위", "monthly_income_hint": "월 소득 약 450~540만원" },
11     { "decile": 8, "label": "8분위", "monthly_income_hint": "월 소득 약 540~660만원" },
12     { "decile": 9, "label": "9분위", "monthly_income_hint": "월 소득 약 660~850만원" },
13     { "decile": 10, "label": "10분위", "monthly_income_hint": "월 소득 약 850만원~" }
14   ]
15 }

```

6.4 2026년 기준 중위소득

shared/median_income_2026.json

24 lines · JSON

```

1  {
2    "year": 2026,
3    "unit": "KRW/month",
4    "source": {
5      "issuer": "보건복지부",
6      "notice_no": "제2025-135호",
7      "title": "2026년 기준 중위소득 및 생계·의료급여 선정기준과 최저보장수준",
8      "url": "https://www.law.go.kr/LSW/admRulLsInfoP.do?admRulId=37680&efYd=0",
9      "effective_from": "2026-01-01",
10     "retrieved_at": "2026-08-08",
11     "_verification": "값 검증은 web/src/lib/median-income.test.mjs 가 2025년 표와 발표 인상률로 대조한다. 7인 가구 값은 고시 산정식과 40,035원 차이가 있어 원문 대조가 필요하다 - 같은 파일의 skip 된 테스트 주석 참조.",
12     "_update_cycle": "연 1회. 매년 7~8월 중앙생활보장위원회 의결 후 고시되며, 다음 해 1월 1일부터 적용된다."
13   },
14   "amounts": {
15     "1": 2564238,
16     "2": 4199292,
17     "3": 5359036,
18     "4": 6494738,
19     "5": 7556719,
20     "6": 8555952,
21     "7": 9515150
22   },
23   "additional_member_amount": 959198
24 }

```

7. 이용 준수사항

항목	준수 내용
출처 표시	모든 결과 화면에 데이터 출처 기관명, 원문 URL, 수집 시각을 표시합니다. 별도의 출처 안내 페이지를 운영합니다.
호출 부하	1일 1회 심야 배치. 이용자 요청 시점에는 공공데이터 API를 호출하지 않습니다. 현재 페이지 요청은 순차 실행하며 별도 1초 throttle은 두지 않습니다.
재시도	408/429/5xx에 한해 최대 2회, 지수 백오프. 4xx는 즉시 중단하여 무효 요청을 반복하지 않습니다.
인증키 관리	소스코드·저장소에 키를 포함하지 않습니다. 환경변수로만 주입하며 공개 저장소에 커밋되지 않습니다.
데이터 가공	원문을 임의로 수정하지 않습니다. 요약을 생성하는 경우에도 원문 링크를 항상 병기하여 이용자가 원본을 확인할 수 있게 합니다.
개인정보	회원가입·로그인이 없습니다. 입력받은 인적사항은 이용자 브라우저 쿠키에만 보관하며 서비스 데이터베이스에 저장하지 않습니다. 공공API로 개인정보를 전송하지 않습니다.
영리성	비영리. 공모전 출품 및 공익 목적이며 광고·유료화 계획이 없습니다.
장애 대응	수집 실패 시 직전 성공 데이터를 유지합니다. 빈 결과를 노출하지 않습니다.

본 문서에 포함된 소스코드와 데이터는 github.com/ofseo03/amuguna 에서 확인하실 수 있습니다.
문의: ofseo03@gmail.com